

FOR_02 — Computer Hardware, File Systems and System Artifacts

Covers TP-I Unit 4 (Introduction to Computer Hardware) and Unit 5 (Windows / Linux / macOS System Artifacts)

Plus pointers to the CORE files for TP-I Units 6, 8 and 9

This is the **densest and highest-yield block in Technical Paper-I**. Unit 5 in particular is pure "you either know the artifact or you don't" material — there is nothing to reason out, and every fact you memorise converts directly into marks. A CS graduate walks into this with a large head start on the *hardware* half (§1–§4) and almost none on the *artifacts* half (§5–§8). Budget your time accordingly: skim §1–§4, grind §5–§8.

Reading order suggestion: §1 → §2 → §3 → §4 (one sitting, they are familiar) → then three separate passes over §5 (Windows). §6 (Linux) and §7 (macOS) are one sitting each.

§1. Computer Hardware from a Forensic Angle

1.1 Concept — why an examiner cares about hardware

An ordinary IT technician looks at hardware and asks "*does it work?*". A forensic examiner looks at the same hardware and asks three completely different questions:

1. **Where can data hide in this thing?** (Every chip that retains state is a potential exhibit — not just the hard disk, but RAM, the BIOS/UEFI NVRAM chip, the SSD's over-provisioned area, the printer's internal memory, the router's flash, the GPU's VRAM.)
2. **How fast will that data disappear if I do the wrong thing?** (The **order of volatility** — see [FOR_03](#) §4.3. Pulling power destroys RAM; booting the machine destroys timestamps.)
3. **How do I get a bit-for-bit copy out without changing a single bit?** (Write blockers, interfaces, adapters, encryption.)

Everything in this section should be read through those three questions.

1.2 The major components and their forensic significance

Component	Function (one line)	Forensic significance
Motherboard	The main printed circuit board carrying CPU, chipset, RAM slots, expansion slots and connectors; hosts all buses	Bears the BIOS/UEFI firmware chip and the CMOS/NVRAM holding date-time, boot order and passwords. Model/serial identifies the machine. Firmware can be maliciously modified (bootkits)
CPU / Processor	Executes instructions; ALU + CU + registers + cache	Holds transient data in registers/cache — the most volatile evidence of all , effectively unrecoverable. CPU features matter (AES-NI for full-disk encryption, TPM support, virtualisation extensions)
Chipset (Northbridge/ Southbridge, or modern PCH)	Routes traffic between CPU, memory and I/O	Determines available interfaces (SATA, NVMe, USB) — dictates which write blocker/adaptor you need

RAM (main memory)	Volatile working memory	Gold-mine evidence : running processes, network connections, open files, clipboard, injected malware, encryption keys and passphrases in plaintext , decrypted document contents, chat fragments. Lost on power-off (though see cold-boot attacks)
ROM / BIOS-UEFI chip	Firmware that initialises hardware and starts the boot	Boot order, secure-boot state, supervisor password, and possible firmware implants
CMOS + battery	Small NVRAM holding BIOS settings and the real-time clock	System date/time offset — critical, because every timestamp on the disk must be interpreted relative to the machine's clock and time zone. Always record BIOS time vs true time at seizure
Storage devices	HDD, SSD, USB, SD, optical, tape	The primary evidence container. See §2
GPU / video card	Rendering	VRAM may hold recently displayed screen content (research-level, rarely used in casework)
Power supply (SMPS)	Converts mains to DC	Not evidence, but note wattage/connectors so you can power the machine in the lab if needed
Expansion cards	NIC, sound, RAID controller, capture cards	A hardware RAID controller matters enormously: a single disk pulled from a RAID set is meaningless without the controller's stripe/parity parameters
Peripherals	Keyboard, mouse, printer, scanner	Hardware keyloggers hide in keyboard connectors. Printers retain spooled jobs in internal storage; some print near-invisible yellow tracking dots (MIC — Machine Identification Code)
Networking components	NIC, switch, router, modem, Wi-Fi AP	MAC addresses, ARP/DHCP tables, connection logs, port-forward rules, Wi-Fi client history. Routers hold volatile logs lost on reboot

Memory hierarchy (know this ordering — MCQ material)

Registers → L1 cache → L2 cache → L3 cache → Main memory (RAM)
→ SSD / HDD (secondary) → Optical / tape / cloud (tertiary, offline)

Going down: capacity ↑ cost per bit ↓ access time ↑ volatility ↓

Volatile vs non-volatile: volatile memory (SRAM cache, DRAM main memory) loses contents when power is removed; non-volatile memory (ROM, EPROM, EEPROM, flash, magnetic disk, optical) does not. **Note the trap**: *flash memory is non-volatile, and cache is volatile*.

1.3 Networking components — quick forensic table

Device	Layer	What an examiner can get from it
NIC	1–2	MAC address (the machine's L2 identity); promiscuous-mode capability implies sniffing
Hub	1	Broadcasts to all ports — trivially sniffable (historically important; now obsolete)
Switch	2	MAC/CAM table showing which MAC was on which port and when; requires port mirroring / SPAN to sniff
Router	3	Routing table, NAT translation table , ARP cache, DHCP leases, firewall/ACL logs, port forwards, connection logs — mostly volatile, lost on reboot

Firewall / UTM	3–7	Allow/deny logs, IDS alerts, VPN tunnel records
Wireless AP	1–2	Associated client MACs, SSID, encryption mode, DHCP leases, connection times
Modem / ONT	1	ISP account linkage — the bridge from a public IP to a subscriber
Proxy / server logs	7	URL-level browsing history for a whole organisation

Exam link: "IP address identifies the *network interface on the internet*; MAC address identifies the *physical adapter on the local segment*." Public IP → ISP → subscriber is the normal tracing route; MAC is only useful within the local broadcast domain (and is trivially spoofed).

§2. Storage Devices — Geometry, and Why SSDs Broke Forensics

2.1 Hard Disk Drive (HDD) — the classical model

The intuition

An HDD is a stack of spinning metal/glass **platters** coated with a magnetic film, with a read/write **head** floating a few nanometres above each surface on a cushion of air. Data is written as tiny magnetised regions. Because the platters spin and the head swings on an arm, the data is naturally organised into concentric rings and pie-slices.

The geometry vocabulary — learn all six

Term	Definition	Note
Platter	A single rigid disk; both surfaces are used	A drive has 1–5 platters typically
Head	One read/write head per usable surface	2 heads per platter
Track	One concentric ring on one surface	Numbered from 0 at the outer edge inward
Cylinder	The set of all tracks at the same radius across all platters	Reading a whole cylinder needs no head movement — hence the old CHS scheme
Sector	The smallest physically addressable unit	Classically 512 bytes ; modern drives use 4096 bytes ("Advanced Format" / 4Kn), often with 512e emulation
Cluster (allocation unit)	The smallest unit the <i>file system</i> will allocate to a file = 1, 2, 4, 8 ... sectors	This is where file slack comes from — see §5.9

Addressing: CHS vs LBA

- **CHS (Cylinder–Head–Sector)** — the old scheme; an address is a triple (*C, H, S*). Limited by BIOS field widths (the famous 504 MB and 8.4 GB barriers).
- **LBA (Logical Block Addressing)** — the modern scheme; the drive is presented as a simple **linear array of sectors numbered 0, 1, 2, ...** and the drive's own firmware translates to physical geometry. **All modern forensic imaging works in LBA terms.** LBA-48 supports 2^{48} sectors $\approx$ 128 PiB.

The fact that the *drive firmware* does the translation is itself forensically important: the examiner never sees true physical geometry, and the firmware can hide areas from the host (see HPA/DCO below).

Hidden areas on an HDD — very examinable

Area	What it is	Forensic point
HPA — Host Protected Area	A region at the end of the disk hidden from the OS via the ATA <code>SET MAX ADDRESS</code> command; originally for recovery partitions	A suspect can hide data here. A good imaging tool/write blocker must detect and unlock the HPA and image it
DCO — Device Configuration Overlay	Makes the drive report <i>fewer</i> sectors/features than it has	Same risk; can coexist with an HPA. Must be detected and removed for a complete image
G-list / P-list (remapped sectors)	Sectors retired by the drive firmware as bad; their old contents remain on the platter	Data in remapped sectors is invisible to normal imaging — recoverable only with vendor/firmware-level tools
Service area / SA	Firmware modules and translator tables stored on the platters outside the user LBA range	Advanced/lab-level acquisition only

Magnetic recording notes

- **PMR/CMR** (perpendicular/conventional) vs **SMR** (shingled). SMR drives overlap tracks like roof shingles; rewriting one track disturbs neighbours, so the drive does read-modify-write via a media cache — this makes SMR behaviour, and therefore deleted-data persistence, less predictable.
- The old idea that data can be recovered from a **single-pass overwritten** modern drive by magnetic force microscopy is **not supported in practice**. A single full overwrite is effectively destructive on modern densities. (Multi-pass standards such as the old DoD 5220.22-M 7-pass scheme are historical. ⚠ verify before quoting standard numbers.)

2.2 Solid State Drives (SSD) — and the forensic problem

The intuition

An SSD has no moving parts. It stores bits as trapped charge in **NAND flash** cells. Flash has two awkward physical properties that change everything:

1. **You can write a page (4–16 KB) but you can only ERASE a whole block (128 pages+)**. So an SSD can never overwrite data in place the way an HDD does; it writes the new version to a fresh page and marks the old page invalid.
2. **Each flash block survives only a limited number of erase cycles** (a few thousand for TLC/QLC).

To cope, the SSD's **controller** runs a **Flash Translation Layer (FTL)** that continuously moves data around behind the host's back. Three mechanisms matter:

Mechanism	What it does	Why it breaks forensics
Wear levelling	Spreads writes evenly across all blocks, relocating even static data	The physical location of data is decoupled from its LBA. Old copies of "overwritten" files linger in retired physical pages that the host can never address — so evidence may exist but be unreachable; conversely, a "wiped" file may still exist

TRIM / UNMAP / Deallocate	The OS tells the SSD "these LBAs belong to a deleted file"; the controller then erases those blocks during idle time	This is the killer. On a TRIM-enabled SSD, deleted-file content is often zeroed within seconds or minutes, without any user action — so classical deleted-file recovery and carving frequently fail entirely
Garbage collection	Background consolidation of valid pages and erasure of stale blocks	The drive changes its own contents while merely powered on , even with a write blocker attached. Two images of the same untouched SSD can therefore have different hash values
Over-provisioning	7–28% of the NAND is reserved and never exposed as LBAs	Contains data the host cannot address without chip-off/ vendor tools
Compression / dedup (some controllers)	Data stored transformed	Physical NAND contents do not resemble logical contents
Self-encrypting drives (SED/ OPAL)	Data always encrypted with an internal media key	A "secure erase" merely destroys the key — instant, irreversible wipe of the entire drive

The exam-ready statement (write this verbatim)

On an SSD, the drive's own controller modifies the stored data autonomously (through TRIM, garbage collection and wear levelling), even when the drive is merely powered on and connected through a hardware write blocker. This breaks two foundational assumptions of digital forensics: (a) that deleted data persists until overwritten by the user, and (b) that repeated acquisitions of an unaltered device yield identical hash values. Consequently, deleted-file recovery and file carving on a TRIM-enabled SSD are unreliable, and any hash mismatch between two acquisitions must be documented and explained rather than treated as evidence of tampering.

Practical countermeasures

- Image the SSD **as early as possible**, before idle time lets garbage collection run.
- Prefer acquiring while the system is **live** if the data of interest is in the file system cache or memory.
- Document the drive model, firmware and TRIM support in the report; **anticipate the hash question in cross-examination**.
- Chip-off / vendor-mode acquisition can reach over-provisioned NAND, but the FTL mapping must then be reconstructed — specialist work.
- TRIM is **not** issued in some situations: some USB/UAS enclosures, some RAID configurations, older OS/file-system combinations, and encrypted volumes configured without TRIM pass-through. In those cases classical recovery may still work — always test, never assume.

2.3 Other storage media

Medium	Notes for the examiner
USB flash drive	Same NAND issues but usually no TRIM over USB Mass Storage → deleted-file recovery often still works. Records leave USB history in the Windows registry (§5.4)
SD / microSD	Wear levelling present, TRIM usually absent. Very common in mobile and camera cases
eMMC / UFS	Soldered flash in phones and tablets → chip-off or ISP/JTAG for physical acquisition
Optical (CD/ DVD/Blu-ray)	Write-once media are naturally tamper-evident; check for multi-session discs where later sessions hide earlier ones

§3. The Boot Process, Step by Step

3.1 Why an examiner must know this cold

Two reasons, and both are exam-worthy:

- 1. **Accidentally booting the evidence machine destroys evidence.** Windows boot alone will modify hundreds of files, write to the registry, update the last-access and last-boot timestamps, run scheduled tasks, create prefetch entries, mount and possibly *repair* the file system, and may trigger anti-forensic logic-bombs. Understanding the boot chain tells you exactly what gets written and when.
- 2. **Malware lives in the boot chain.** Boot-sector viruses, MBR bootkits, UEFI implants and malicious bootloaders all sit at points in this sequence.

3.2 The sequence — legacy BIOS path

```
1. POWER ON → PSU asserts "power good"
  ↓
2. CPU resets, begins execution at the reset vector → firmware (BIOS) in ROM
  ↓
3. POST (Power-On Self-Test)
  • tests CPU, RAM, video, keyboard, buses
  • errors reported by beep codes / POST codes (video may not be up yet)
  ↓
4. BIOS initialises hardware, reads settings from CMOS/NVRAM
  • boot device order, system date/time, passwords
  ↓
5. BIOS loads the first 512-byte sector (LBA 0) of the boot device = the MBR
  • checks the boot signature 0x55AA at offset 510-511
  ↓
6. MBR bootstrap code (446 bytes) reads the partition table (4 × 16-byte entries)
  • finds the partition marked ACTIVE
  • loads that partition's VBR (Volume Boot Record / boot sector)
  ↓
7. VBR loads the OS bootloader
  • Windows: BOOTMGR → reads BCD store → WINLOAD.EXE
  • Linux: GRUB stage 1.5/2 → reads grub.cfg → loads vmlinuz + initrd
  ↓
8. Kernel loads, initialises drivers, mounts the root file system
  ↓
9. First user-space process starts
  • Windows: SMSS → CSRSS → WININIT → SERVICES.EXE / LSASS → WINLOGON → USERINIT → Explorer
  • Linux: init / systemd → targets/runlevels → getty/display manager
  ↓
10. User login → user profile loaded (NTUSER.DAT hive on Windows) → shell
```

3.3 UEFI path — the modern one

UEFI (Unified Extensible Firmware Interface) replaces BIOS. Key differences:

Aspect	Legacy BIOS + MBR	UEFI + GPT
Firmware interface	16-bit real mode, interrupt-based	32/64-bit, modular, driver-based

usually changes only the *bookkeeping*, not the *data*.

4.2 FAT family

FAT = File Allocation Table. The disk is divided into clusters; a table at the start of the volume has one entry per cluster, and the entry holds the **number of the next cluster in the file** — so a file is stored as a **linked list threaded through the FAT**. The last cluster's entry holds an end-of-chain marker; free clusters hold 0.

Volume layout (FAT):

Boot sector (VBR)	FAT #1	FAT #2 (copy)	Root directory (FAT12/16 only)	Data area (clusters 2, 3, 4 ...)
----------------------	--------	------------------	-----------------------------------	-------------------------------------

Directory entry (32 bytes): name(8) ext(3) attr(1) ... first-cluster ... size(4)

Chain: dir entry → cluster 5 → FAT[5]=6 → FAT[6]=9 → FAT[9]=0xFFFF (end)

What happens on deletion in FAT (classic exam answer):

1. The **first character of the file name in the directory entry is replaced by 0xE5** (the sigma-like character shown as ? by DOS).
2. All the **FAT entries in the file's cluster chain are set to 0 (free)**.
3. **The file's data clusters are untouched.**

→ Recovery: the directory entry still gives the **starting cluster and size**; if the file was **contiguous** and nothing has overwritten it, it can be recovered completely. If it was **fragmented**, the chain is gone and recovery beyond the first fragment requires carving. This "lost chain" problem is precisely why FAT recovery of fragmented files is unreliable — a very common 5-mark question.

Variant	Cluster address bits	Max volume (typical)	Max file size	Notes
FAT12	12	~16 MB	—	Floppies; still used on very small volumes
FAT16	16	2 GB (4 GB with 64 KB clusters)	2 GB	Older DOS/Windows; fixed-size root directory (512 entries)
FAT32	28 (32 with 4 reserved)	2 TB (Windows formats up to 32 GB)	4 GB – 1 byte	Universal interchange format; root directory is a normal cluster chain; no permissions, no journaling
exFAT	32	128 PB (theoretical)	16 EB (theoretical)	Designed for flash media/SDXC ; no journaling by default (has an optional TexFAT); uses an allocation bitmap plus FAT only for fragmented files; supports timestamps with UTC offsets

The 4 GB limit on FAT32 is a guaranteed MCQ. It is why large video files cannot be copied to an ordinary FAT32-formatted USB stick, and why SDXC cards ship as exFAT.

4.3 NTFS — the one to know in depth

NTFS = New Technology File System. Its central idea: **everything on the volume, including the file system's own metadata, is a file** — and all of them are described in one master table.

The MFT (Master File Table)

- The **MFT** is a file (`$MFT`) containing one **record per file/directory**, typically **1024 bytes** each.
- Each record is a set of **attributes**, the important ones being:

Attribute	Contains
<code>\$STANDARD_INFORMATION</code> (<code>\$SI</code>)	The four MACE timestamps, DOS-style file attributes, owner/security ID, quota. Writable via Windows API → this is what timestamping tools alter
<code>\$FILE_NAME</code> (<code>\$FN</code>)	File name (and a second set of MACE timestamps). Normally only the kernel updates these , so a mismatch between <code>\$SI</code> and <code>\$FN</code> timestamps is a strong timestamping indicator
<code>\$DATA</code>	The actual file content. An unnamed default stream plus any number of named streams → this is the mechanism behind Alternate Data Streams (§5.7)
<code>\$INDEX_ROOT</code> / <code>\$INDEX_ALLOCATION</code>	B-tree index used for directory contents
<code>\$BITMAP</code>	Cluster allocation bitmap for the volume (<code>\$Bitmap</code>) or index
<code>\$SECURITY_DESCRIPTOR</code>	ACL (older NTFS; modern versions centralise in <code>\$Secure</code>)

- **Resident vs non-resident:** a file small enough (roughly < **700–900 bytes** of data) is stored **inside its own MFT record** — "resident". Larger files store **cluster runs** pointing to the data area — "non-resident". **Forensic consequence:** a deleted small file may survive entirely inside the MFT even after its data area is reused; conversely, tiny files leave no trace in the data area at all.

NTFS metadata files (the `$` files, MFT records 0–15)

Record	Name	Purpose
0	<code>\$MFT</code>	The MFT itself
1	<code>\$MFTMirr</code>	Backup copy of the first few MFT records
2	<code>\$LogFile</code>	Transaction journal — a forensic goldmine of recent file operations
3	<code>\$Volume</code>	Volume label, version, dirty flag
4	<code>\$AttrDef</code>	Attribute definitions
5	<code>.</code>	The root directory
6	<code>\$Bitmap</code>	Cluster allocation bitmap — tells you which clusters are "free" (i.e. unallocated space)
7	<code>\$Boot</code>	The VBR / boot sector, as a file
8	<code>\$BadClus</code>	Bad cluster list — can be abused to hide data
9	<code>\$Secure</code>	Security descriptors
10	<code>\$UpCase</code>	Uppercase conversion table
11	<code>\$Extend</code>	Directory holding <code>\$UsnJrnl</code> , <code>\$Quota</code> , <code>\$ObjId</code> , <code>\$Reparse</code>

`$UsnJrnl` (Update Sequence Number Journal, in `$Extend`) records every change to every file — creation, deletion, rename, data overwrite — with a reason code. Combined with `$LogFile`, it lets an examiner **reconstruct a timeline of file activity even for files that**

no longer exist. Name these two in any NTFS answer.

Other NTFS features with forensic weight

Feature	Forensic relevance
Journaling (<code>\$LogFile</code>)	Recent metadata transactions recoverable; also means the file system self-repairs, which can alter evidence if you mount it read-write
Alternate Data Streams	See §5.7
Hard links, junctions, symbolic links, reparse points	One data set, many names; watch for double-counting or for a path that redirects elsewhere
Sparse files	Unwritten regions consume no clusters and read as zeros
Compression (LZNT1)	Content is compressed per 16-cluster unit — keyword searches on the raw disk will miss compressed content
EFS (Encrypting File System)	Per-file encryption tied to the user's certificate/DPAPI keys; the Data Recovery Agent and the user's master key are the routes in
Volume Shadow Copies (VSS)	Point-in-time snapshots. One of the most productive artifacts in Windows forensics — old versions of files, deleted files and earlier registry hives can be recovered from shadow copies
ADS on directories	Streams can attach to folders too, not only files
Quotas, Object IDs	Object ID (<code>\$objId</code>) can link a file to a specific machine (contains a MAC address in the older GUID scheme)

4.4 ext2 / ext3 / ext4 (Linux)

Core concept: **the inode**. See §6.2 for the full treatment. Briefly:

Feature	ext2	ext3	ext4
Journaling	No	Yes	Yes
Extents (vs block pointers)	No	No	Yes — better for large files, less fragmentation
Max file size	2 TB	2 TB	16 TB
Max volume size	32 TB	32 TB	1 EB
Delayed allocation	No	No	Yes
Nanosecond timestamps	No	No	Yes (plus a creation time field, <code>crtime</code> , not exposed by <code>stat</code> in all versions)
Deleted-file recovery	Comparatively easy — block pointers often survive	Harder	Hardest — ext3/ext4 zero the block pointers / extent tree in the inode on delete

Key exam fact: ext3 and ext4 wipe the inode's data-block references when a file is deleted, so classical "undelete via inode" fails and you must rely on **carving** or on the journal. ext2 does not, which is why old Linux textbooks make undeletion sound easy.

Other Linux/Unix file systems worth naming: **XFS**, **Btrfs** (copy-on-write, snapshots — good

for forensics because old versions persist), **ZFS**, **ReiserFS** (historical), **F2FS** (flash-friendly, used on some Android devices), and **JFS**.

4.5 HFS+ and APFS (macOS)

Aspect	HFS+ ("Mac OS Extended", 1998)	APFS (Apple File System, 2017)
Core structure	Catalog File — a B-tree of all files and folders, keyed by Catalog Node ID (CNID)	Container holding multiple volumes that share a free-space pool
Journaling	Yes (HFS+J)	Metadata via copy-on-write, no traditional journal
Copy-on-write	No	Yes — never overwrites live data in place
Snapshots	No (Time Machine used hard links)	Yes, native — highly valuable evidence source
Encryption	FileVault (whole disk, CoreStorage)	Native per-file and per-volume encryption (FileVault 2)
Timestamps	1-second resolution	Nanosecond
Space sharing	Fixed partitions	Volumes grow/shrink dynamically inside a container
Cloning	No	Instant file clones (shared blocks)
Forensic note	Deleted-file recovery reasonably tractable	Copy-on-write means old versions may persist , but encryption-by-default and the container layout complicate acquisition; tool support is still less mature

Other Apple structures to name: **Extents Overflow File**, **Attributes File** and **Allocation File** in HFS+; **resource forks** (the historical Mac analogue of NTFS ADS); the **.DS_Store** file in every browsed folder (records folder view settings — proves a folder was opened).

4.6 File-system comparison table — reproduce this in the exam

File system	Native OS	Max file size	Max volume	Journaling	Permissions/ACL	Encryption	Key forensic point
FAT12	DOS	32 MB	~16 MB	X	X	X	Floppies; 12-bit FAT entries
FAT16	DOS/Win	2 GB	2–4 GB	X	X	X	Fixed root directory
FAT32	Win 95 OSR2+	4 GB – 1	2 TB (32 GB via Windows format)	X	X	X	Deletion sets first name byte to 0xE5 and frees the chain
exFAT	Win/ macOS/ Android	16 EB	128 PB	X (optional TexFAT)	X	X	Default for SDXC / large flash ; interchange format

NTFS	Windows NT+	16 TB–8 PB (cluster dependent)	8 PB	✓ \$LogFile	✓ ACLs	✓ EFS/ BitLocker	MFT , ADS, \$UsnJrnl, Volume Shadow Copies, compression
ReFS	Windows Server	very large	very large	✓ (integrity streams)	✓	✓	No ADS support in early versions; limited tool support
ext2	Linux	2 TB	32 TB	✗	✓ (POSIX)	✗	Easiest Linux undelete — inode pointers survive
ext3	Linux	2 TB	32 TB	✓	✓	✗	Inode block pointers zeroed on delete
ext4	Linux	16 TB	1 EB	✓	✓ (+ACL)	✓ (fscrypt)	Extents; nanosecond + creation timestamps; carving usually required
XFS / Btrfs / ZFS	Linux/ Unix	very large	very large	✓ / CoW	✓	✓	Btrfs/ZFS snapshots preserve deleted data
HFS+	macOS ≤10.12	8 EB	8 EB	✓	✓	✓ (FileVault)	Catalog B-tree , resource forks, .DS_Store
APFS	macOS 10.13+, iOS	8 EB	8 EB	CoW	✓	✓ native, per-file	Containers, snapshots , clones, encryption by default
F2FS	Android/ Linux	3.94 TB	16 TB	✓	✓	✓	Log-structured, flash-aware; used on many Android devices
ISO 9660 / UDF	Optical	—	—	✗	✗	✗	Multi-session discs may hide earlier sessions

§4.7 Likely exam questions

Marks	Question
5	What is a file system? Name any four and state one distinguishing feature of each.
5	What happens when a file is deleted from a FAT32 volume?

5	What is the MFT? What information does it hold?
5	Distinguish between ext2, ext3 and ext4.
10	Compare FAT32 and NTFS from a forensic point of view.
10	Explain the structure of NTFS and list its metadata files with their forensic value.
15	Discuss the major file systems used by Windows, Linux and macOS. Compare them in a table and explain how the choice of file system affects the recovery of deleted data.

MCQ traps

- FAT32 max **file** size is 4 GB – 1 byte; max **volume** is 2 TB. Don't confuse the two.
- NTFS deletion marks the MFT record unallocated — it does **not** rename the file to `0xE5` (that is FAT).
- **\$LogFile** = **journal**; **\$MFT** = **master file table**; **\$Bitmap** = **cluster allocation**. Constantly swapped.
- **exFAT has no journal** in its standard form.
- **ADS is an NTFS feature**, not FAT/exFAT — copying an ADS-bearing file to a FAT32 stick **silently destroys the stream** (this is also a practical anti-forensic/detection point).
- APFS is **copy-on-write**; HFS+ is not.

§5. Windows System Artifacts — the core of TP-I

Unit 5

Everything below answers one question: **"What did the user do on this machine, and when?"**

Windows is extraordinarily talkative. It records program execution, device attachment, folder browsing, file opening, searching and network connections — mostly for performance and convenience reasons, without the user's knowledge. Each of those records is an artifact.

Organising principle for the exam: group artifacts by the *question they answer*.

Investigative question	Artifacts that answer it
What programs were run?	Prefetch, UserAssist, ShimCache/AppCompatCache, Amcache, SRUM, Jump Lists, <code>MUICache</code> , Windows Event Log 4688
What files were opened?	RecentDocs, LNK files, Jump Lists, Office MRU, <code>OpenSavePidlMRU</code> , thumbnail cache
What folders were browsed?	ShellBags , LNK files, <code>.lnk</code> in Recent
What devices were attached?	<code>USBSTOR</code> , <code>USB</code> , <code>MountedDevices</code> , <code>MountPoints2</code> , <code>setupapi.dev.log</code> , Event Log 20001
Who logged in, when, from where?	Security event log (4624/4625/4634/4648/4672), <code>SAM</code> hive, <code>RDP</code> logs
What was typed / searched / opened in dialogs?	<code>RunMRU</code> , <code>TypedPaths</code> , <code>TypedURLs</code> , <code>WordWheelQuery</code> , <code>OpenSaveMRU</code>
What survives deletion?	Recycle Bin <code>\$I/\$R</code> , file slack, unallocated space, <code>\$LogFile</code> , <code>\$UsnJrnl</code> , Volume Shadow Copies, pagefile, hiberfil
What network did it join?	<code>NetworkList</code> profiles, DHCP settings, Wi-Fi profiles, Event Log

slack is a real and examinable concept.

5.3 Key forensic registry keys — the master table

This is the single most memorisation-heavy table in the whole elective, and it is worth doing. A 10- or 15-mark "discuss the forensic value of the Windows registry" answer that reproduces even eight of these rows with the right hive will score at or near full marks.

(a) System and configuration

Purpose	Key
Computer name	SYSTEM\CurrentControlSet\Control\ComputerName\ComputerName
Time zone (essential for interpreting all timestamps)	SYSTEM\CurrentControlSet\Control\TimeZoneInformation
Which ControlSet is current	SYSTEM\Select → Current, LastKnownGood
OS version, build, install date, registered owner	SOFTWARE\Microsoft\Windows NT\CurrentVersion
Last shutdown time	SYSTEM\CurrentControlSet\Control\Windows → ShutdownTime (binary FILETIME)
Installed applications	SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall
Services and drivers	SYSTEM\CurrentControlSet\Services
User profile list (SID → username → profile path)	SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList
Whether prefetch is enabled	SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PrefetchParameters
Whether last-access timestamps are disabled	same FileSystem key → NtfsDisableLastAccessUpdate (disabled by default since Vista — an important caveat when interpreting access times)
ClearPageFileAtShutdown (anti-forensic setting)	SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management

(b) USB device history — a guaranteed question

What you learn	Key / file

Every USB mass-storage device ever attached: vendor, product, revision and serial number: each device is its own subkey, so the key LastWrite ≈ first/last connection	<code>SYSTEM\CurrentControlSet\Enum\USBSTOR\Disk&Ven_X&Prod_Y&Rev_Z\<SerialNumber></code>
All USB devices including non-storage (VID/PID)	<code>SYSTEM\CurrentControlSet\Enum\USB\VID_xxxx&PID_yyyy\<serial></code>
First install, last connected, last removed timestamps per device	<code>SYSTEM\CurrentControlSet\Enum\USBSTOR\...</code> <code>\Properties\{83da6326-97a6-4088-9453-a1923f573b29}\0064 / 0066 / 0067</code> (△ verify GUID digits before quoting; the concept — that per-device install/first-connect/last-connect/last-removal timestamps live under <code>Properties</code> — is the examinable part)
Volume GUID / drive-letter mapping for the device	<code>SYSTEM\MountedDevices</code> (<code>\DosDevices\E:</code> and <code>\??\Volume{GUID}</code> entries, whose data contains the device signature)
Which user used the device	<code>NTUSER.DAT</code> → <code>Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2\{Volume GUID}</code> — matching the GUID from <code>MountedDevices</code> attributes the device to a specific user account
Human-readable device names	<code>SYSTEM\CurrentControlSet\Enum\USBSTOR</code> <code>FriendlyName</code> , and <code>SOFTWARE\Microsoft\Windows Portable Devices\Devices</code> (holds the volume label)
First-ever installation time, driver install log	<code>C:\Windows\INF\setupapi.dev.log</code> (<code>setupapi.log</code> on XP) — text file, easy to read, gives the first connection date/time
Live/removal events	Event log <code>Microsoft-Windows-DriverFrameworks-UserMode/Operational</code> (IDs 2003, 2100, 2102) and <code>Partition/Diagnostic</code> (ID 1006) △ verify IDs

The classic 10-mark walkthrough — "prove that a specific USB stick was connected by a specific user":

1. `USBSTOR` → find the device by vendor/product; record the **serial number**.

2. `USBSTOR\...\Properties` → first-install and last-connected timestamps.
3. `setupapi.dev.log` → corroborate the **first** connection date.
4. `MountedDevices` → find the **Volume GUID** and the drive letter assigned.
5. `NTUSER.DAT\...\MountPoints2\{that GUID}` in a **specific user's** hive → attributes it to **that user**; the key `LastWrite` gives the time.
6. `Windows Portable Devices\Devices` → volume label, to match the physical exhibit.
7. Corroborate with **LNK files and Jump Lists** whose target path is on that volume (they store the volume serial number and label), and with **ShellBags** for folders browsed on it.
8. Corroborate with the **event log** entries above.

Notice the shape of that answer: *artifact* → *what it proves* → *corroboration*. Use that shape for every "how would you prove X" question in this paper.

(c) MRU (Most Recently Used) lists — user activity

All of these live in `NTUSER.DAT` under

`Software\Microsoft\Windows\CurrentVersion\Explorer\:`

Key	What it records
RecentDocs	Recently opened documents , sub-keyed by file extension; holds an <code>MRUListEx</code> ordering
RunMRU	Everything typed into the Start → Run dialog (a favourite of intruders using <code>cmd, \\server\share</code>)
TypedPaths	Paths typed into the Explorer address bar
<code>ComDlg32\OpenSavePidlMRU</code>	Files chosen in Open/Save common dialogs, grouped by extension
<code>ComDlg32\LastVisitedPidlMRU</code>	The application used and the folder it last used for Open/Save
WordWheelQuery	Terms typed into the Windows Explorer / Start-menu search box — shows what the user was <i>looking for</i> , often the most probative artifact of all
RecentApps (Win10-era)	Recently used applications and the files they opened △ verify presence per build
<code>Map Network Drive MRU</code>	Network shares mapped by the user
<code>TypedURLs</code> (under <code>Software\Microsoft\Internet Explorer\</code>)	URLs typed into IE/legacy Edge — typed, not merely visited

Application MRUs are equally valuable: `Software\Microsoft\Office\<ver>\<App>\File MRU`, media-player recent files, archive-tool recent paths, and so on.

(d) Run keys / persistence — malware and autostart

Key	Behaviour
<code>SOFTWARE\Microsoft\Windows\CurrentVersion\Run</code>	Runs at every logon (all users)
<code>SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce</code>	Runs once, then the value is deleted
<code>NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Run / RunOnce</code>	Same, per user

... \Policies \Explorer \Run	Policy-driven autostart
SOFTWARE \Microsoft \Windows NT \CurrentVersion \Winlogon → Shell, Userinit	Hijacked by malware — should be explorer userinit.exe respectively; anything appended suspicious
SYSTEM \CurrentControlSet \Services	Services and drivers with Start=2 (auto) or Start=0 (boot/system driver)
SOFTWARE \Microsoft \Windows NT \CurrentVersion \Image File Execution Options \<exe> → Debugger	Classic IFEO hijack — launches an attacker binary when the named exe runs (also the "sticky keys" backdoor)
AppInit_DLLs, AppCertDlls, KnownDLLs	DLL injection persistence
Startup folders (not registry)	%AppData% \Microsoft \Windows \Start Menu \Programs \Startup (per user); %ProgramData% \Startup (all users)
Scheduled Tasks (not registry)	C: \Windows \System32 \Tasks \ (XML files) SOFTWARE \Microsoft \Windows NT \CurrentVersion \Schedule \TaskCache
WMI event subscriptions	C: \Windows \System32 \wbem \Repository \Cimv2 \Subscriptions — fileless persistence

(e) UserAssist — proof of GUI program execution

- **Location:** NTUSER.DAT \Software \Microsoft \Windows \CurrentVersion \Explorer \UserAssist \{GUID} \Count
- **What it records:** programs the user launched **through the Windows GUI** (Explorer, Start menu, desktop shortcuts), with a **run count**, a **last-execution timestamp** and (on later versions) a focus-time value.
- **The famous quirk:** value names are **ROT-13 encoded** (each letter rotated 13 places) — purely as light obfuscation. Tools decode automatically. *This single fact appears in MCQ sets constantly.*
- **Limitation to state in the answer:** UserAssist records **GUI-launched** programs only. Anything started from a command line, a script, a service or a scheduled task **will not appear** — so UserAssist absence is not evidence of non-execution. Corroborate with Prefetch, ShimCache and Amcache.
- Common GUIDs: {CEBFF5CD-...} for executables and {F4E57C4B-...} for shortcut links
⚠ verify the full GUIDs before writing them out; naming the *concept* suffices.

(f) ShellBags — proof that a folder was browsed

- **Concept:** Windows remembers, per user, the **view settings** (icon size, sort order, column layout, window position) of every folder you open in Explorer — so the folder looks the same next time. To do that it must record **that the folder was opened**, and it keeps that record **even after the folder is deleted or the removable drive is gone**.
- **Location:**
- Windows 7 and later: UsrClass.dat → Local Settings \Software \Microsoft \Windows \Shell \BagMRU and ... \Shell \Bags
- Also NTUSER.DAT \Software \Microsoft \Windows \Shell \BagMRU / Bags
- Windows XP: NTUSER.DAT \Software \Microsoft \Windows \ShellNoRoam \BagMRU

- **Forensic value — say all four:**
 1. Proves a **specific user** browsed a **specific folder path**, with a timestamp.
 2. **Survives deletion** of the folder and disconnection of the device — evidence of folders on **removable media, network shares, encrypted containers (TrueCrypt/VeraCrypt volumes) and ZIP archives** that no longer exist or are not present.
 3. Reconstructs a **directory tree** the examiner has never seen, from names alone (useful when the content is gone but the folder names — "Invoices_2025", "backup_of_client_db" — are themselves probative).
 4. The BagMRU structure preserves the **hierarchy** and MRU **ordering** of browsing.
- **Limitation:** ShellBags record folders opened in **Explorer/dialog windows**; folders touched only by the command line generally do not appear.

(g) Network and miscellaneous

Purpose	Key
Networks the machine joined (SSID/domain, first and last connected time , network type)	SOFTWARE\Microsoft\Windows NT\CurrentVersion\NetworkList\Profiles\{GUID} and ... \Signatures\Unmanaged (holds the gateway MAC , which geolocates the connection)
Static/DHCP IP, DHCP server, lease time per interface	SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces\{GUID}
ShimCache / AppCompatCache (evidence of executables present on the system, with file path, size and last-modified time; presence, not necessarily execution)	SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache
Amcache (installed/executed programs, SHA-1 hash of the binary , path, publisher) — a <i>file</i> , not part of the standard hives	C:\Windows\AppCompat\Programs\Amcache.hve
MUICache (application names of executed programs)	UsrClass.dat\Local Settings\...\MuiCache
BAM/DAM (Background/Desktop Activity Moderator) — last execution time per executable per user SID	SYSTEM\CurrentControlSet\Services\bam\State\UserSettings\<SID> ⚠ verify path per build

SRUM (System Resource Usage Monitor — per-application network bytes sent/received , useful for proving data exfiltration)	<code>C:\Windows\System32\SRU\SRUDB.dat</code> (ESE database, not registry)
Recently mounted/known volumes	<code>SYSTEM\MountedDevices</code>
Autologon credentials (sometimes plaintext!)	<code>SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon</code> → <code>DefaultUserName</code> , <code>DefaultPassword</code>

The "program execution" family is a favourite comparison question. Learn the distinction:

Artifact	Proves	Gives run count?	Gives timestamps?
Prefetch	Execution	✓	Last 8 run times + first run
UserAssist	GUI execution by that user	✓	Last run
ShimCache	Presence (and possibly execution)	✗	File last-modified; cache entry order
Amcache	Presence/installation + SHA-1 hash	✗	First-seen
BAM/DAM	Execution per user	✗	Last run
SRUM	Execution + network bytes	—	Hourly buckets
Event 4688	Process creation (if auditing enabled)	✗	Exact time + command line

5.4 Windows Event Logs

Concept

Windows records system, security and application events in structured logs. Since Vista these are `.evtx` files (XML-based binary); XP and earlier used `.evt`.

Location: `C:\Windows\System32\winevt\Logs\*.evtx`
(XP: `C:\Windows\System32\config\*.evt`)

The three classic logs, plus the modern additions:

Log	Contents
Security.evtx	Logons, logoffs, privilege use, object access, policy change, account management — only what auditing is configured to record
System.evtx	Driver/service events, boot and shutdown, hardware, time changes
Application.evtx	Application-generated events, crashes
<code>Setup.evtx</code>	Servicing/updates
<code>Microsoft-Windows-TerminalServices-*/Operational</code>	RDP connections (source IP, username)

Microsoft-Windows-Windows Defender/ Operational	Malware detections
Microsoft-Windows-PowerShell/ Operational (4103, 4104)	Script block logging — the actual PowerShell commands run
Microsoft-Windows-TaskScheduler/ Operational	Task creation/execution
Microsoft-Windows-DriverFrameworks- UserMode/Operational	USB insert/remove
Microsoft-Windows-Bits-Client/ Operational	BITS transfers — used for exfiltration

Key event IDs — memorise the starred ones

ID	Log	Meaning
4624 ★	Security	Successful logon (check the Logon Type)
4625 ★	Security	Failed logon — repeated 4625s = brute force
4634 / 4647	Security	Logoff / user-initiated logoff
4648	Security	Logon using explicit credentials (runas) — lateral movement indicator
4672	Security	Special privileges assigned to a new logon (administrative logon)
4720 / 4726	Security	User account created / deleted
4722 / 4725	Security	Account enabled / disabled
4724 / 4723	Security	Password reset by admin / changed by user
4728 / 4732 / 4756	Security	Member added to a global / local / universal security group (privilege escalation)
4688 ★	Security	New process created (with command line if the policy is enabled)
4689	Security	Process exited
4663 / 4656 / 4660	Security	Object access attempted / handle requested / object deleted (needs SACL auditing)
4698 / 4702	Security	Scheduled task created / updated
4697	Security	Service installed — a classic malware persistence indicator
1102 ★	Security	The audit log was cleared — a top-tier anti-forensic indicator
104	System	An event log was cleared (System log analogue of 1102)
7045 ★	System	A new service was installed

7034 / 7035 / 7036	System	Service crashed / control sent / entered running-stopped state
6005 / 6006	System	Event log service started / stopped — proxy for boot / clean shutdown
6008	System	Unexpected shutdown (crash or power loss)
6013	System	System uptime (daily)
1074	System	Shutdown/restart initiated, with the user and reason
41	System	System rebooted without a clean shutdown (Kernel-Power)
1 (Sysmon)	Sysmon	Process creation with hashes — only if Sysmon is installed
4776	Security	NTLM credential validation
5140 / 5145	Security	Network share accessed / detailed share access
1149	TerminalServices-RemoteConnectionManager	RDP authentication succeeded (gives the source IP)
21 / 24 / 25	TerminalServices-LocalSessionManager	RDP session logon / disconnect / reconnect
20001	System (DriverFrameworks/UMPNP)	Device driver installation — USB attachment

⚠ Event ID meanings occasionally shift across Windows versions and some IDs above appear in multiple providers. The starred ones are stable and safe to quote. Verify anything unusual against Microsoft documentation rather than memory.

Logon Types in event 4624 — a very common sub-question

Type	Meaning
2	Interactive — logged on at the physical console
3	Network — accessed a share or service over the network (most common)
4	Batch — scheduled task
5	Service — a service started with these credentials
7	Unlock — workstation unlocked
8	NetworkCleartext — credentials sent in the clear (e.g. basic auth)
9	NewCredentials — <code>runas /netonly</code>
10	RemoteInteractive — RDP / Terminal Services
11	CachedInteractive — logged on with cached domain credentials (laptop off-network)

Anti-forensics note: event logs are a prime target for tampering. Watch for **1102/104** (log cleared), for **gaps in the record-number sequence**, for a suspiciously small `.evtx` file, and for a log whose earliest entry post-dates the incident. Deleted event records can often be carved from unallocated space, the pagefile and Volume Shadow Copies.

5.5 LNK (Shortcut) Files

Concept

A `.lnk` file is a Windows Shell Link — a small binary file that points at a target. Windows **creates them automatically** whenever a user opens a document, without the user ever making a shortcut. The crucial property: **the LNK file records information about the target that survives the target's deletion, and even the removal of the entire drive it lived on.**

Automatic locations:

- `C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\` — one LNK per recently opened file
- `...\Recent\AutomaticDestinations\` and `...\CustomDestinations\` — Jump Lists (see §5.6)
- `...\Start Menu\`, Desktop, Startup folder — user-created and installer-created shortcuts
- `...\Microsoft\Office\Recent\` — Office recent files

What a LNK file contains — list at least six in the exam:

Field	Value
Target full path	Including the drive letter and folder structure
Target's MAC timestamps	Created / Modified / Accessed of the target file at the time the LNK was last updated
Target file size	In bytes
Volume information	Volume serial number , volume label, and drive type (fixed / removable / network / CD-ROM)
Network share path	For files opened over a UNC path
NetBIOS name and MAC address of the machine (in the <code>TrackerDataBlock</code>)	Links the file to a specific machine — extremely powerful when the LNK is found on <i>another</i> computer
Icon location, working directory, command-line arguments, window state	Also present
The LNK file's own MAC timestamps	Its creation time ≈ first time the target was opened; its modification time ≈ most recent time

The killer exam sentence: "A LNK file in the Recent folder proves that a specific file at a specific path, on a specific volume (identified by serial number), of a specific size, was opened by that user at a specific time — and this remains provable even though the file itself and the USB drive it was on are long gone."

5.6 Prefetch and Jump Lists

Prefetch — evidence of execution

- **Purpose (non-forensic):** Windows monitors the first ~10 seconds of an application's start-up, records which files and DLLs it loads, and stores that map so the next launch is faster.
- **Location:** `C:\Windows\Prefetch\`
- **Naming:** `EXECUTABLENAME-XXXXXXXXX.pf` where `XXXXXXXXX` is a **hash of the full path (and, on Win10+, command-line arguments)**.

→ **Two .pf files for the same exe name mean the program was run from two different paths** — a strong indicator that a legitimate-looking binary (e.g. `svchost.exe`) was executed from an unusual location.

- **What each .pf contains:**
- **Run count**
- **Last run time** — and on **Windows 8 and later, the last EIGHT run timestamps**
- **First run time** (from the .pf file's own creation date)
- **The full path of the executable**
- **A list of files and directories the program referenced** (DLLs, config files, and often the documents it opened)
- **Volume information and volume serial numbers** — including of **removable drives**
- **Default limits:** 128 entries on Windows 7 and earlier; **1024 entries on Windows 8+**. Once full, the oldest are discarded.
- **Configuration:** `EnablePrefetcher` in `...\Memory Management\PrefetchParameters` (0 = off, 1 = app only, 2 = boot only, 3 = both). **Prefetch is disabled by default on SSDs in some configurations and on Windows Server** — a caveat worth stating in the answer.
- **Superfetch/SysMain** .db files sit in the same folder and hold related data.

The classic use case: a suspect denies ever running a data-wiping tool. `CCLEANER.EXE-XXXX.pf` shows a run count of 4 with the last run 11 minutes before seizure, and the referenced-file list includes the paths of the very directories now empty. That is a complete narrative from one file.

Jump Lists — recent items per application

- **Concept:** the "recent files" list you see when you right-click a taskbar icon. Windows stores it on disk, per application, in the **OLE Compound File (structured storage)** format — each stream inside is effectively a **LNK file**.
- **Locations:**
- `%AppData%\Microsoft\Windows\Recent\AutomaticDestinations\<AppID>.automaticDestinations-ms`
→ generated **automatically** by Windows as the user opens files
- `%AppData%\Microsoft\Windows\Recent\CustomDestinations\<AppID>.customDestinations-ms`
→ items the **application** or the user **pinned**
- **AppID** is a hash identifying the application; published AppID lists let you say *which* program the list belongs to.
- **Forensic value:** file paths, timestamps, volume serial numbers and access order **per application**, surviving deletion of the files — and the `DestList` stream inside gives an **MRU ordering with access counts**. Especially valuable for proving what was opened from a removable drive or a network share.

5.7 Alternate Data Streams (ADS) — a guaranteed topic

The intuition

On NTFS, a file's content is stored in a `$DATA` attribute. The **default, unnamed \$DATA** stream is what you see when you open the file. But NTFS allows **any number of additional, named \$DATA streams attached to the same file**. Windows Explorer shows only the unnamed stream, and **the file's reported size counts only the unnamed stream**. So you can hide an entire executable inside a 0-byte text file and Explorer will insist the file is 0 bytes.

Historically ADS existed for **Macintosh interoperability** — HFS files had a data fork and a **resource fork**, and NTFS needed somewhere to put the resource fork when serving files to Mac clients via Services for Macintosh.

Syntax

```
filename:streamname:$DATA      ← the general form

C:\> echo secret text > report.txt:hidden.txt
C:\> type nc.exe > innocent.doc:nc.exe
C:\> dir                        ← report.txt shows 0 bytes; the stream is invisible
C:\> dir /R                     ← Vista+ : LISTS THE STREAMS
C:\> more < report.txt:hidden.txt
PS> Get-Item report.txt -Stream *      ← PowerShell
PS> Get-Content report.txt -Stream hidden.txt
C:\> wmic process call create "C:\path\innocent.doc:nc.exe"  ← older execution trick
```

Legitimate uses (mention these — it shows understanding)

Stream	Purpose
:Zone.Identifier	The Mark of the Web — written by browsers and mail clients to record that a file came from the internet. Contains <code>ZoneId=3</code> (Internet), and on modern Windows also the <code>HostUrl</code> and <code>ReferrerUrl</code> . Forensically superb: it proves a file was downloaded, and from where
:favicon	Site icons
:SummaryInformation, : {4c8cc155-...}	Document metadata, "encrypted" folder marker
:AFP_AfpInfo, :AFP_Resource	Macintosh resource forks

Why it matters forensically

1. **Data hiding / anti-forensics** — files, tools and stolen data concealed from casual inspection.
2. **Malware concealment and execution** — payloads stored in a stream of a benign file.
3. **The Zone.Identifier stream is positive evidence of download** — arguably the *most* useful ADS.
4. **Detection is easy once you know:** `dir /R`, PowerShell `Get-Item -Stream *`, Sysinternals `streams.exe`, or any forensic tool that parses the MFT (every stream is visible in the MFT record).
5. **ADS is NTFS-only. Copying the file to FAT32/exFAT, emailing it, or uploading it strips the streams silently** — so an examiner must work from the forensic image, not from a copied file, and a suspect who moved data to a USB stick may have destroyed their own hidden streams.

§5.7.1 Exam-ready model answer skeleton for "Write a note on Alternate Data Streams"

1. Definition — additional named `$DATA` attributes in an NTFS MFT record.
2. Origin — Macintosh resource-fork compatibility.
3. Syntax `file:stream` with a command example.

4. Invisibility — not shown by Explorer/`dir`; size not counted.
5. Legitimate use — `Zone.Identifier` / Mark of the Web.
6. Malicious use — hiding tools and data, malware execution.
7. Detection — `dir /R`, `Get-Item -Stream *`, `streams.exe`, MFT parsing.
8. Limitation — NTFS-only; lost on copy to non-NTFS.

5.8 Hidden files and attributes

Mechanism	Description	How the examiner defeats it
File attributes	<code>H</code> (Hidden), <code>S</code> (System), <code>R</code> (Read-only), <code>A</code> (Archive). <code>attrib +h +s secret.txt</code> hides a file even from "show hidden files" unless "hide protected operating system files" is also unticked	The MFT records the attributes plainly; forensic tools ignore them entirely
Alternate Data Streams	§5.7	<code>dir /R</code> , MFT parse
Misleading extension	<code>payload.exe</code> renamed <code>photo.jpg</code>	File signature analysis — see FOR_03 §6.4
Double extension + hidden known extensions	<code>invoice.pdf.exe</code> displays as <code>invoice.pdf</code>	Show extensions; check the signature
Unicode RTLO trick	The Right-to-Left Override character <code>U+202E</code> makes <code>photo\u202Egpj.exe</code> display as <code>photoexe.jpg</code>	Hex view of the file name; tools flag it
Hiding in slack space	§5.9	Slack extraction and keyword search
Hiding in unallocated space / <code>\$BadClus</code> / HPA / DCO	§2.1	Full physical image including HPA/DCO
Steganography	Data embedded inside an image/audio file	Steganalysis — FOR_03 §6.6
Encryption / password-protected archives	Content unreadable	Password recovery, memory analysis for keys, DPAPI, key escrow
Deeply nested / oddly named directories, <code>System Volume Information</code> , <code>\$Recycle.Bin</code>	Social hiding	Full recursive listing from the image
Rootkits	Hooks the OS so the file is invisible even to the OS	Dead-box imaging defeats it entirely — the rootkit is not running

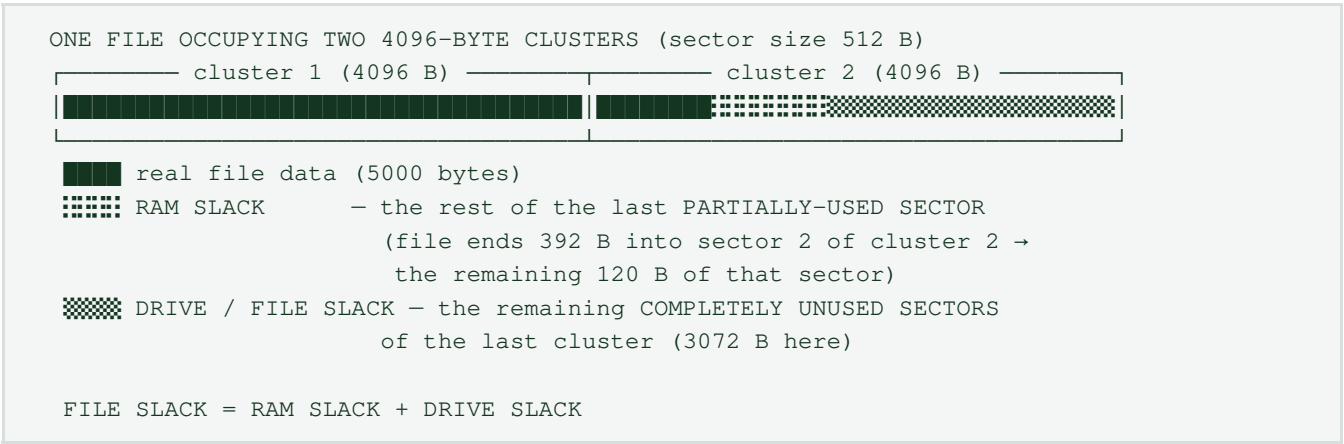
The single most important principle in this whole subsection: *every one of these hiding techniques defeats the operating system's view of the disk. Forensic examination does not use the operating system's view — it parses the raw file-system structures from a bit-stream image. That is why "hidden" is almost meaningless to a forensic examiner.*

5.9 File slack vs volume slack — the guaranteed question

Build the intuition first

The file system allocates space in whole **clusters**. A cluster might be 4096 bytes. If your file is 5000 bytes, the file system must give it **two** clusters = 8192 bytes. The file occupies the first 5000; the remaining **3192 bytes are allocated to the file but not used by it**. That unused tail is **slack**, and it is not blank — **it contains whatever was on those sectors previously**.

Now zoom in. Those 3192 leftover bytes are not all the same:



The three (really four) kinds of slack

Term	Precise meaning	What it contains
RAM slack (a.k.a. sector slack / memory slack)	The space from the end of the file to the end of the last partially used sector	Historically: whatever happened to be in RAM at the moment of the write — hence the name; potentially passwords, fragments of other documents, memory structures. Modern Windows/Linux pad this with zeros , so it is far less productive today than the textbooks suggest — state both facts
Drive slack (a.k.a. file slack proper, cluster slack)	The completely unused sectors between the end of the last used sector and the end of the last cluster	Data from a previously deleted file that occupied those sectors — never overwritten because the OS only wrote as far as the new file's length
File slack	RAM slack + drive slack — the whole gap between end-of-file and end-of-last-cluster	Both of the above
Volume slack (a.k.a. partition slack / drive slack in some texts)	The space between the end of the file system (the last cluster the file system uses) and the end of the partition/ volume itself — arising because the partition size is rarely an exact multiple of the cluster size, or because a volume was resized	Data from a previous, larger file system on that partition — including old file-system metadata, and it is a favourite place to hide data because no file system references it at all

(related) Partition/unpartitioned space	Space on the physical disk not assigned to any partition — e.g. before the first partition, between partitions, or after the last	Old partitions, hidden data, boot code
--	--	--

The comparison table to reproduce

	File slack	Volume slack
Where	Inside an allocated cluster belonging to a live file	Between the end of the file system and the end of the partition
Boundary it arises from	File size vs cluster size	File-system size vs partition size
Owned by	A specific existing file	No file, and no file system
Typical size	0 to (cluster size – 1) per file; huge in aggregate across thousands of files	A few sectors up to many megabytes, once per volume
Typical content	Remnants of previously deleted files; historically RAM contents	Remnants of a previous/larger file system; deliberately hidden data
Visible to the OS?	No — the OS reads only up to the file's logical length	No — outside the file system entirely
How recovered	Forensic tool extracts the bytes beyond logical EOF for every file; keyword-search the slack	Image the whole partition (and the whole physical disk) and examine the tail region
Anti-forensic use	Small payloads hidden in slack (e.g. <code>slacker.exe</code> -style tools)	Larger payloads hidden where no file system will ever touch them

Related: unallocated space

- **Definition:** clusters that the file system's bitmap/FAT marks as **free** — either never used or belonging to **deleted files**.
- **Why it matters:** this is where the *bodies* of deleted files sit until overwritten. It is the primary target of **file carving** (FOR_03 §6.3).
- **Distinguish clearly in the exam:**
- **File slack** = space *inside an allocated cluster* that the live file does not use.
- **Unallocated space** = *whole clusters* not allocated to any live file.
- **Volume slack** = space *inside the partition but outside the file system*.
- **Unpartitioned space** = space *on the disk but outside every partition*.

Practical exam sentence: "A bit-stream (physical) image captures file slack, volume slack, unallocated space and unpartitioned space; a logical copy captures none of them. This is the whole reason forensic imaging must be physical."

Likely exam framing

- 5 marks: "Differentiate between file slack and volume slack." → the table above + the diagram.
- 10 marks: add RAM vs drive slack, unallocated space, and why physical imaging is required.

5.10 The Recycle Bin — \$I and \$R

Concept

Deleting a file in Explorer does not delete it — it **moves** it into a hidden per-user folder and records where it came from, so it can be restored.

Windows version	Folder	Structure
XP and earlier	C:\RECYCLER\<SID>\	Files renamed D<drive><index>.<ext> (e.g. Dc1.txt), with a single hidden index file INFO2 holding the original names, paths, deletion timestamps and sizes for all files
Vista and later	C:\\$Recycle.Bin\<SID>\	Two files per deleted item: • \$I<random6>.<ext> — the metadata: original full path and name, original file size, and the deletion date/time (FILETIME) • \$R<random6>.<ext> — the actual file content, byte-for-byte

The six random characters are the **same** in the matching \$I and \$R pair — that is how the examiner pairs them.

\$I file layout (⚠ **verify exact offsets before quoting them as fact; the *fields* are the examinable part**)

offset 0x00	(8 bytes)	header/version	(1 for Vista/7, 2 for Win10)
offset 0x08	(8 bytes)	original FILE SIZE	in bytes
offset 0x10	(8 bytes)	DELETION timestamp	(Windows FILETIME, UTC)
offset 0x18	...	original FULL PATH and file name	(UTF-16LE) (v2 prefixes this with a 4-byte name length)

Forensic points to make

1. The <SID> sub-folder **attributes the deletion to a specific user account** — look the SID up in SOFTWARE\...\ProfileList.
2. \$I gives the **original path** — proving the file was, e.g., in D:\ClientData\ before deletion.
3. \$I gives the **exact deletion time**, often the most important fact in a spoliation or data-theft case.
4. **Emptying the Recycle Bin** deletes the \$I/\$R pairs, but they then sit in **unallocated space** and are readily **carved** — and \$I files are small, distinctive and easy to find.
5. Files deleted with **Shift+Delete**, from the **command line**, from a **network drive**, from **removable media** (by default), or larger than the bin's quota **never enter the Recycle Bin at all** — so an empty bin proves nothing.
6. \$Recycle.Bin also appears on **each volume**, including attached USB drives.
7. **Volume Shadow Copies** may contain earlier states of the Recycle Bin.

5.11 Thumbnail cache (thumbcache) and other image artifacts

Artifact	Location	Value

Thumbs.db (XP / Win 2000, and still generated for network folders)	In each folder browsed in thumbnail view	Con thur imag may been long the f Bec: lives fold easy over
Thumbcache (Vista+)	C: \\Users\\<user>\\AppData\\Local\\Microsoft\\Windows\\Explorer\\thumbcache_*.db (sizes 32, 96, 256, 1024, 1280, 1920 ...) plus thumbcache_idx.db	Cen thun all fo Prov imag on t and look ever orig dele unre — d CSA imag case
Icon cache	iconcache_*.db in the same folder	Icon insta exec appl evid now prog exisi
WebCacheV01.dat	%LocalAppData%\\Microsoft\\Windows\\WebCache\\	ESE hold Edg cook cach met: usec Winc com
.DS_Store equivalent	—	(mar see

Note the pattern: **caches are built for performance, and performance caches never get cleaned up properly.** That is why they are the examiner's friend.

5.12 Pagefile, swapfile and hiberfil

File	Path	What it is	Forensic value
<code>pagefile.sys</code>	<code>C:\pagefile.sys</code> (hidden, system)	The virtual-memory paging file — pages of RAM written to disk when physical memory is short	A partial snapshot of RAM on disk : fragments of documents, chat messages, browsing content, decrypted data, passwords and encryption keys , malware code. Unstructured — searched by keyword and carving , not parsed
<code>swapfile.sys</code>	<code>C:\swapfile.sys</code>	Used for suspending modern/UWP apps	Same nature, smaller
<code>hiberfil.sys</code>	<code>C:\hiberfil.sys</code>	A compressed image of the entire contents of RAM , written when the machine hibernates (also used by Windows Fast Startup , which hibernates the kernel session on "shutdown")	The best RAM evidence available from a powered-off machine . Can be decompressed and analysed with Volatility (via <code>imagecopy</code> / hibernation plugins) or Hibr2Bin to recover processes, network connections, keys and open documents <i>as they were at hibernation time</i> . Always check for it on a laptop image
Crash dumps	<code>C:\Windows\MEMORY.DMP</code> , <code>C:\Windows\Minidump*.dmp</code>	Kernel or complete memory dumps written on a bugcheck	Same as hiberfil — a memory image from the past
Volume Shadow Copies	<code>\System Volume Information\</code>	Snapshots	Old versions of <i>all</i> of the above

Anti-forensic counterpart: `ClearPageFileAtShutdown = 1` in

`SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` zeroes the pagefile at every shutdown; hibernation can be disabled with `powercfg /h off` (which **deletes hiberfil.sys**).

Finding these settings enabled is itself evidence of anti-forensic awareness — say so in the report.

§5.13 Windows artifacts — the single consolidated location table

If you memorise nothing else in §5, memorise this. It is the answer to "List the important Windows forensic artifacts and their locations."

Artifact	Location
Registry hives (machine)	<code>C:\Windows\System32\config\</code> — <code>SYSTEM</code> , <code>SOFTWARE</code> , <code>SAM</code> , <code>SECURITY</code> , <code>DEFAULT</code> (+ <code>.LOG*</code>)
Registry hive (per user)	<code>C:\Users\<u>\NTUSER.DAT</code>

Registry hive (per user, ShellBags)	C:\Users\<u>\AppData\Local\Microsoft\Windows\UsrClass.dat
Event logs	C:\Windows\System32\winevt\Logs*.evtx
Prefetch	C:\Windows\Prefetch*.pf
Amcache	C:\Windows\AppCompat\Programs\Amcache.hve
SRUM	C:\Windows\System32\SRU\SRUDB.dat
Recent LNK files	C:\Users\<u>\AppData\Roaming\Microsoft\Windows\Recent\
Jump Lists	...\Recent\AutomaticDestinations\ and ...\CustomDestinations\
Recycle Bin	C:\\$Recycle.Bin\<SID>\\$I..., \$R...
Thumbnail cache	C:\Users\<u>\AppData\Local\Microsoft\Windows\Explorer\thumbcache_*.db
Scheduled tasks	C:\Windows\System32\Tasks\
Startup folder	C:\Users\<u>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\
Hosts file (DNS hijack)	C:\Windows\System32\drivers\etc\hosts
USB driver install log	C:\Windows\INF\setupapi.dev.log
Pagefile / swapfile / hiberfil	C:\pagefile.sys, C:\swapfile.sys, C:\hiberfil.sys
Volume Shadow Copies	\System Volume Information\
Windows Search index	C:\ProgramData\Microsoft\Search\Data\Applications\Windows\Windows.edb
Browser data	see FOR_03 §8
WMI repository	C:\Windows\System32\wbem\Repository\OBJECTS.DATA
User profiles	C:\Users\<u>\ (Desktop, Documents, Downloads, AppData{Roaming,Local,LocalLow})

§5.14 Likely exam questions (Windows artifacts)

Marks	Question
5	What is the Windows registry? Name the five root keys.
5	List the registry hive files and their locations on disk.
5	What is an Alternate Data Stream? How is it detected?
5	Differentiate between file slack and volume slack.
5	What is a prefetch file? What forensic information does it provide?

5	Explain the <code>\$I</code> and <code>\$R</code> files of the Windows Recycle Bin.
5	What is UserAssist? Why are its entries ROT-13 encoded?
5	What are ShellBags and what do they prove?
5	State the forensic value of <code>pagefile.sys</code> and <code>hiberfil.sys</code> .
10	Discuss the forensic importance of the Windows registry, listing at least eight keys of evidentiary value.
10	Describe the Windows event log. List ten important event IDs with their meaning.
10	What is a LNK file? Explain how LNK files can prove access to a file on a removable drive that is no longer available.
10	Explain slack space, unallocated space and volume slack with a diagram, and their role in evidence recovery.
15	"Windows records far more than the user realises." Discuss the principal Windows system artifacts, their on-disk locations and the investigative questions each answers.
15	An employee is suspected of copying confidential files to a USB drive and deleting the originals. Describe, artifact by artifact, how you would prove this from a forensic image of his Windows workstation.

MCQ traps (Windows artifacts)

- **NTUSER.DAT** is per user; **SYSTEM/SOFTWARE/SAM** are machine-wide in `System32\config`.
- **HKEY_LOCAL_MACHINE\HARDWARE** has no file on disk — it is rebuilt at every boot.
- Registry **keys** have LastWrite times; **values** do not.
- **UserAssist** = ROT-13. (Not Base64, not XOR.)
- **Prefetch proves execution; ShimCache proves presence.**
- Prefetch limit: 128 (Win7) → **1024 (Win8+)**; **last 8 run times on Win8+**, only 1 on Win7.
- **ShellBags live mainly in `UsrClass.dat` on Windows 7+.**
- **`$I` holds the metadata, `$R` holds the data.** Constantly swapped in options.
- XP used `RECYCLER` + a single `INFO2` file; Vista+ uses `$Recycle.Bin` + `$I/$R`.
- **ADS is NTFS-only** and is destroyed by copying to FAT.
- **RAM slack is to the end of the SECTOR; drive/file slack is to the end of the CLUSTER.**
- **`hiberfil.sys` = compressed RAM image; `pagefile.sys` = virtual memory overflow.**
- **Event 1102 = security log cleared; 4624 = successful logon; 4625 = failed logon; 7045 = new service installed.** These four are the most-swapped.
- Logon **type 10 = RDP, type 2 = interactive console, type 3 = network.**

§6. Linux System and Artifacts

6.1 File system layout — the Filesystem Hierarchy Standard

Linux presents **one single tree rooted at `/`**; every device is *mounted* somewhere inside it (there are no drive letters). Learn the directories and, for each, **the forensic reason you would go there.**

Directory	Purpose	Forensic interest
<code>/</code>	Root of everything	—
<code>/bin</code> , <code>/sbin</code> , <code>/usr/bin</code> , <code>/usr/sbin</code>	Essential and system binaries	Trojaned system binaries (a classic rootkit) — verify against package hashes (<code>rpm -Va</code> , <code>debsums</code>)
<code>/boot</code>	Kernel (<code>vmlinuz</code>), <code>initrd</code> , GRUB config	Bootkits; <code>grub.cfg</code> shows boot parameters
<code>/dev</code>	Device nodes	<code>/dev/sda</code> , <code>/dev/sda1</code> naming; <code>/dev/null</code> , <code>/dev/random</code>
<code>/etc</code>	All system configuration	<code>passwd</code> , <code>shadow</code> , <code>group</code> , <code>sudoers</code> , <code>hosts</code> , <code>hostname</code> , <code>fstab</code> , <code>crontab</code> , <code>resolv.conf</code> , <code>ssh/</code> , <code>network/</code> . The first place to look
<code>/home/<user></code>	User home directories	User data, dot-files (§6.4), <code>.bash_history</code> , <code>.ssh/</code>
<code>/lib</code> , <code>/lib64</code> , <code>/usr/lib</code>	Shared libraries	LD_PRELOAD rootkits, malicious <code>.so</code> files
<code>/media</code> , <code>/mnt</code>	Mount points for removable and temporary media	Evidence that a USB device was mounted
<code>/opt</code>	Third-party software	Non-package-managed (often unauthorised) applications
<code>/proc</code>	Virtual file system exposing kernel/process state — exists only in RAM	On a live system: <code>/proc/<pid>/cmdline</code> , <code>/exe</code> , <code>/fd</code> , <code>/maps</code> , <code>/net/tcp</code> . Nothing here appears in a dead-box image — a key live-forensics point
<code>/root</code>	The root user's home	Root's <code>.bash_history</code> — often the whole intrusion story
<code>/sys</code>	Virtual kernel/device interface	Live only
<code>/tmp</code> , <code>/var/tmp</code>	Temporary files	Attackers' favourite working directory (world-writable). <code>/tmp</code> is often cleared at boot or is a tmpfs (RAM) — <code>/var/tmp</code> survives reboot
<code>/usr</code>	User programs, docs, source	<code>/usr/local/</code> for locally compiled binaries
<code>/var</code>	Variable data	/var/log (§6.6), <code>/var/spool/{mail,cron,cups}</code> , <code>/var/www</code> (web content and upload directories — webshells), <code>/var/lib</code> (databases, package state, Docker)

6.2 Inodes — the central concept

Intuition

In Linux, **the file name is not part of the file**. A file *is* an **inode**: a fixed-size structure holding all the metadata plus pointers to the data blocks. A **directory is simply a file containing a list of (name → inode number) pairs**. This decoupling explains hard links, explains why you can delete an open file, and explains why deleted-file recovery works the way it does.

What an inode contains

Field	Note
Inode number	Unique within the file system (not across file systems)
File type and permission bits (mode)	Regular / directory / symlink / block / char / FIFO / socket
UID and GID	Owner and group (numeric)
Size	In bytes
Link count	How many directory entries point to this inode
Timestamps	atime (last access), mtime (last data modification), ctime (last inode/metadata change) — and in ext4 also crtime (creation)
Pointers to data blocks	Direct, single-, double- and triple-indirect (ext2/3) or an extent tree (ext4)

The MAC times question, done right:

Time	Changes when	Can the user set it?
atime	The file's contents are read	Yes (<code>touch -a</code>) — and it is often disabled by the <code>noatime/relatime</code> mount option for performance, so treat it with caution
mtime	The file's contents are written	Yes (<code>touch -m</code>)
ctime	The inode changes — a write, a rename, a permission or ownership change, a link count change. Also updated whenever mtime is changed	No — not directly settable by any normal utility. Only by changing the system clock or by raw disk editing
crtime (ext4)	File creation	Not exposed by <code>stat</code> on all systems; readable with <code>debugfs -R 'stat <inode>' /dev/sdaX</code>

Therefore: an mtime that is older than expected while the ctime is recent is a strong indicator of timestamping. This is the Linux analogue of the NTFS `$SI` vs `$FN` check. Say this in any anti-forensics answer.

Deletion in Linux

1. The **directory entry** is removed (the name → inode link).
2. The inode's **link count** is decremented; at zero (and with no process holding it open) the inode and its data blocks are marked **free** in the inode bitmap and block bitmap.
3. **ext2**: the block pointers in the inode usually survive → tools like `debugfs` and `ext3grep/extundelete` can often recover the file.
4. **ext3/ext4**: the block pointers / extent tree in the inode are **zeroed** → the map from inode to data is destroyed. Recovery relies on **the journal** (which may hold an older copy of the inode) or on **carving** the raw blocks.

```
- rwx r-x r-- 1 alice devs 4096 Sep 5 09:12 report.txt
↑   ↑   ↑   ↑       ↑       ↑
|   |   |   ^ other    |       ^ group
|   |   ^ group      ^ owner (user)
|   ^ owner
^ file type: - regular d directory l symlink
              b block dev c char dev p FIFO s socket
```

The three special bits — very examinable:

Bit	Octal	On a file	On a directory
SUID (Set User ID)	4000	The program runs with the file owner's privileges , not the caller's. Shown as s in the owner's execute position (<code>rwsr-xr-x</code>)	(no effect)
SGID (Set Group ID)	2000	Runs with the file's group privileges	New files inherit the directory's group
Sticky bit	1000	(obsolete)	Only the owner of a file may delete it , even in a world-writable directory. This is why <code>/tmp</code> is <code>drwxrwxrwt</code>

Forensic red flag, and a guaranteed exam point: an attacker who obtains root frequently leaves a **SUID-root shell** as a backdoor (e.g. a copy of `bash` with mode 4755 owned by root hidden in `/tmp` or `/dev/shm`). **Hunting command:**

```
find / -perm -4000 -type f -exec ls -l {} \; (or find / -perm -u=s).
```

Compare the result against the known-good SUID list for the distribution.

Compare the result against the known-good SUID list for the distribution.

Also mention: **ACLs** (`getfacl/setfacl`) beyond the classic bits; **extended attributes** (`getfattr, xattr`) which can carry SELinux labels and even user-defined data — **a Linux data-hiding location roughly analogous to NTFS ADS**; **capabilities** (`getcap`) as a finer-grained alternative to SUID; **immutable flag** `chattr +i` (a file that even root cannot delete until `chattr -i`) — attackers use it to protect their backdoors, and `lsattr` reveals it.

6.4 Dot-files (hidden files)

- **Any file or directory whose name begins with a `.` is hidden** from `ls` (but not from `ls -a`). That is the *entire* mechanism — there is **no hidden attribute** in Unix. It is a **convention implemented in the listing tool, not in the file system**. State this explicitly; it contrasts nicely with the Windows `H` attribute.
- **Attackers exploit it** with names like `...`, `.` (dot-space), `..` variants and `.hidden_dir` which are visually easy to overlook even in `ls -a`.
- **High-value dot-files in a user's home directory:**

Dot-file	Contains
<code>.bash_history</code>	Command history — see §6.7
<code>.bash_profile</code> , <code>.bashrc</code> , <code>.profile</code>	Startup scripts — persistence (aliases hiding commands, malicious lines appended at the end)
<code>.ssh/</code>	<code>authorized_keys</code> (an attacker's public key here = a permanent backdoor), <code>known_hosts</code> (which hosts this user connected TO — pivot evidence; may be hashed), <code>id_rsa/id_ed25519</code> private keys, <code>config</code>
<code>.gnupg/</code>	PGP keys
<code>.viminfo</code> , <code>.lessht</code> , <code>.mysql_history</code> , <code>.python_history</code> , <code>.psql_history</code>	Other command/file histories that users forget to clear — frequently more useful than <code>.bash_history</code>
<code>.local/share/</code> , <code>.config/</code>	Application data (browsers, chat clients, recently-used file lists — <code>.local/share/recently-used.xbel</code>)
<code>.thumbnails/</code> or <code>.cache/</code> <code>thumbnails/</code>	Thumbnails of deleted images — the Linux analogue of thumbcache
<code>.Trash/</code> / <code>.local/share/Trash/</code> {files,info}	The Linux "Recycle Bin" : <code>info/*</code> . <code>trashinfo</code> files hold the original path and deletion date , <code>files/</code> holds the content — the direct analogue of <code>\$I/\$R</code>
<code>.recently-used</code> , <code>.gtk-</code> <code>bookmarks</code>	Recent documents, bookmarked folders
<code>.wget-hjsts</code> , <code>.lessht</code>	Evidence of downloads and file viewing

6.5 User accounts — `/etc/passwd`, `/etc/shadow`, `/etc/group`

`/etc/passwd` — world-readable, seven colon-separated fields

```
alice : x : 1001 : 1001 : Alice Kumar,,, : /home/alice : /bin/bash
1      2      3      4      5      6      7

1 username
2 password placeholder - 'x' means the hash is in /etc/shadow
                        (a hash actually present here = a legacy/misconfigured system;
                        an EMPTY field = NO PASSWORD REQUIRED - a serious finding)
3 UID    - 0 = root; 1-999 typically system accounts; 1000+ normal users (Debian/
Ubuntu)
                        500+ on older RHEL
4 GID    - primary group
5 GECOS  - comment field: real name, office, phone
6 home directory
7 login shell - /bin/bash, /bin/sh, or /usr/sbin/nologin / /bin/false for service
accounts
```

Two forensic checks every examiner performs on `/etc/passwd`:

1. **Any account other than `root` with UID 0** — that is a hidden root-equivalent backdoor account. `awk -F: '$3==0 {print $1}' /etc/passwd` should print only `root`.
2. **A service account that has been given a real login shell** (e.g. `www-data` with `/bin/bash`), or a newly added account with an innocuous name (`mysqld`, `sysadm`, `nfsd`).

`/etc/shadow` — root-readable only, nine fields

```
alice : $6$salt$salt$hashhashhash... : 19876 : 0 : 99999 : 7 : : :
 1           2           3         4         5         6 7 8 9

1 username
2 PASSWORD HASH    $1$ = MD5,   $2a$/$2y$ = Blowfish/bcrypt,   $5$ = SHA-256,
                    $6$ = SHA-512,   $y$ = yescrypt (modern default)
                    '*' or '!' = login disabled;  '!' prefix = account LOCKED;
                    EMPTY = no password required
3 date of last password change    (days since 1 Jan 1970)
4 minimum days before the password may be changed again
5 maximum days the password is valid
6 warning period (days)
7 inactivity period after expiry
8 account expiry date (days since epoch)
9 reserved
```

Forensic use: field 3 dates the last password change — **which can date an account compromise**. The hash format tells you what cracking effort is required. The lock/disable markers show which accounts were deliberately disabled and when.

`/etc/group` — four fields

```
devs : x : 1001 : alice,bob,carol
 1     2     3         4

1 group name    2 password placeholder    3 GID    4 supplementary members
```

Check for unauthorised members of `sudo`, `wheel`, `admin`, `adm`, `docker` (membership of `docker` is effectively root) and `disk`.

Related account/authorisation files

- `/etc/sudoers` and `/etc/sudoers.d/` — who may run what as root. **`user ALL=(ALL) NOPASSWD: ALL` appended by an attacker is a classic persistence entry**. Read with `visudo -c` / directly from the image.
- `/etc/login.defs`, `/etc/securetty`, `/etc/pam.d/` — authentication policy; **malicious PAM modules** are a stealthy backdoor.
- `/etc/ssh/sshd_config` — `PermitRootLogin`, `Port`, `AuthorizedKeysFile`, `PasswordAuthentication`.
- `/etc/hosts` — **static DNS overrides used to hijack traffic**.
- `/etc/fstab` — what is mounted where, including hidden/encrypted volumes and network shares.
- `/etc/rc.local`, `/etc/init.d/`, `/etc/systemd/system/*.service`, `~/.config/systemd/user/` — **startup persistence**.

6.6 Logs — `/var/log`

Linux logs are **plain text** (traditional syslog) or **binary** (`wtmp/btmp/lastlog`, and systemd's journal). Know which is which — an MCQ favourite.

File	Format	Contents	Read with
<code>/var/log/auth.log</code> (Debian/ Ubuntu) <code>/var/log/secure</code> (RHEL/ CentOS/Fedora)	Text	Authentication: SSH logins (success and failure), <code>sudo</code> use, <code>su</code> , account creation, PAM messages. The single most important log in an intrusion case	<code>cat</code> , <code>grep</code>
<code>/var/log/syslog</code> (Debian) <code>/var/log/messages</code> (RHEL)	Text	General system messages from all daemons	<code>cat</code> , <code>grep</code>
<code>/var/log/kern.log</code>	Text	Kernel messages — module loads (rootkit LKMs), OOM kills, USB device insertion, firewall drops	
<code>/var/log/wtmp</code>	Binary	All successful logins and logouts , plus boots and shutdowns	<code>last</code>
<code>/var/log/btmp</code>	Binary	Failed login attempts — brute force evidence	<code>lastb</code>
<code>/var/log/lastlog</code>	Binary	The most recent login time for <i>each</i> user	<code>lastlog</code>
<code>/var/run/utmp</code>	Binary	Currently logged-in users (live only)	<code>who</code> , <code>w</code>
<code>/var/log/faillog</code>	Binary	Failed-login counters per user	<code>faillog</code>
<code>/var/log/dmesg</code>	Text	Kernel ring buffer from boot	<code>dmesg</code>
<code>/var/log/cron</code> or <code>/var/log/syslog</code>	Text	Cron job execution	
<code>/var/log/apache2/{access,error}.log</code> , <code>/var/log/nginx/*</code>	Text	Web server logs — the first place to find a webshell upload or SQL-injection attempt (see <code>FOR_03</code> §2.5)	
<code>/var/log/mysql/</code> , <code>/var/log/postgresql/</code>	Text	Database logs	
<code>/var/log/audit/audit.log</code>	Text	auditd — the richest source if enabled: syscalls, file access, <code>execve</code> with arguments	
<code>/var/log/apt/history.log</code> , <code>/var/log/dpkg.log</code> , <code>/var/log/yum.log</code>	Text	Software installed and when — shows an attacker installing tools	
<code>/var/log/journal/</code>	Binary	systemd journal — supersedes/duplicates much of the above on modern systems	<code>journalctl</code> (<code>journalctl --file=...</code> against a mounted image)

<code>/var/log/boot.log, /var/log/Xorg.0.log</code>	Text	Boot and X server	
<code>/var/spool/mail/<user>, /var/mail/</code>	Text (mbox)	Local mail — including cron output , which often reveals what a scheduled job did	

Log rotation: older logs are rotated to `auth.log.1`, `auth.log.2.gz` ... by `logrotate` (configured in `/etc/logrotate.conf`, `/etc/logrotate.d/`). **Always decompress and examine the rotated archives** — the attacker may only have cleaned the current file.

Anti-forensics on Linux logs: attackers use `> /var/log/auth.log` (truncate), `shred`, or purpose-built log-cleaners that surgically remove `wtmp/utmp` entries. Detect by: timeline gaps; a log file whose **inode ctime is much later** than its last entry; a log whose **size is zero** but which the daemon is still writing to; a **missing sequence** in journald; and **corroboration from a remote syslog server**, which is why **centralised remote logging is the standard countermeasure**.

6.7 Bash history

Property	Detail
File	<code>~/.bash_history</code> (also <code>~/.zsh_history</code> , <code>~/.sh_history</code> , <code>~/.history</code>)
When written	Normally only when the shell exits cleanly. An active session's commands live in memory (<code>history</code> shows them) — so killing the terminal or the machine losing power may mean the last session was never written . Conversely, <code>history -a</code> appends immediately
Size	Controlled by <code>HISTSIZE</code> (in-memory) and <code>HISTFILESIZE</code> (on disk)
Timestamps	Present only if <code>HISTTIMEFORMAT</code> was set ; then lines beginning <code>#<epoch></code> precede each command. Usually absent — so bash history normally gives you order but not time
Anti-forensics	<code>unset HISTFILE</code> , <code>export HISTFILE=/dev/null</code> , <code>HISTSIZE=0</code> , <code>history -c</code> , <code>set +o history</code> , prefixing a command with a space (with <code>HISTCONTROL=ignorespace</code>), or <code>ln -s /dev/null ~/.bash_history</code> . Finding any of these in the history itself, or finding the history file symlinked to <code>/dev/null</code>, is itself evidence
Recovery	Deleted/truncated history can be carved from unallocated space ; a copy may exist in a backup, a snapshot, or the swap partition ; <code>root</code> 's history in <code>/root/.bash_history</code> is often forgotten
Other histories	<code>.mysql_history</code> , <code>.psql_history</code> , <code>.python_history</code> , <code>.viminfo</code> (also records files opened in vim, with line positions and search terms), <code>.lessht</code> , <code>.wget-hsts</code> , <code>.node_repl_history</code> — users who clean <code>.bash_history</code> almost never clean these

6.8 Cron and scheduled execution

Location	Contents
<code>/etc/crontab</code>	System-wide crontab — has an extra user field
<code>/etc/cron.d/</code>	Drop-in system cron files (same format as <code>/etc/crontab</code>)

<code>/etc/cron.</code> <code>{hourly,daily,weekly,monthly}/</code>	Scripts run at those intervals by <code>run-parts</code>
<code>/var/spool/cron/crontabs/<user></code> (Debian) <code>/var/spool/cron/<user></code> (RHEL)	Per-user crontabs — created by <code>crontab -e</code> . The classic persistence location
<code>/etc/cron.allow</code> , <code>/etc/cron.deny</code>	Who may use cron
<code>at</code> jobs	<code>/var/spool/at/</code> or <code>/var/spool/cron/atjobs/</code> — one-shot scheduled commands
systemd timers	<code>/etc/systemd/system/*.timer</code> + matching <code>.service</code> ; <code>systemctl list-timers</code> . The modern persistence mechanism that examiners trained only on cron will miss
Anacron	<code>/etc/anacrontab</code> — for machines not always on

Crontab field format (memorise):

```

┌—— minute           (0-59)
| ┌—— hour            (0-23)
| | ┌—— day of month   (1-31)
| | | ┌—— month        (1-12)
| | | | ┌—— day of week (0-7, 0 and 7 = Sunday)
| | | | |
| | | | |
* * * * *      command-to-execute

Example:  */5 * * * * /tmp/.x/beacon.sh    ← "every 5 minutes" — a C2 beacon

```

Forensic reading: a cron entry pointing at a script in `/tmp`, `/dev/shm`, `/var/tmp`, a dot-directory, or a path with a curl/wget pipe to a shell is a near-certain compromise indicator. Cross-check every crontab against `/var/log/cron` (or syslog) for actual execution, and against the referenced script's own timestamps.

§6.9 Likely exam questions (Linux)

Marks	Question
5	What is an inode? What information does it contain?
5	Explain the seven fields of <code>/etc/passwd</code> .
5	Distinguish between <code>atime</code> , <code>mtime</code> and <code>ctime</code> .
5	What is the SUID bit? Why is it a security and forensic concern?
5	How are hidden files implemented in Linux?
5	Distinguish between <code>wtmp</code> , <code>btmp</code> and <code>lastlog</code> .
10	Describe the Linux file-system hierarchy and identify the directories of forensic interest.

10	Discuss the log files found in <code>/var/log</code> and their evidentiary value in investigating an intrusion.
10	Explain Linux file ownership and permissions, including the special bits, with examples.
15	You are given a disk image of a compromised Linux server. Describe systematically the artifacts you would examine and what each would tell you.

MCQ traps (Linux)

- **wtmp** = successful logins (read with **last**); **btmp** = failed (read with **lastb**); **lastlog** = last login per user; **utmp** = currently logged in. The most-swapped set in the paper.
- `/etc/shadow` holds the hashes; `/etc/passwd` holds the `x` placeholder.
- **\$6\$** = SHA-512, **\$1\$** = MD5, **\$2y\$** = bcrypt, **\$5\$** = SHA-256.
- **ctime** is **change time (inode metadata)**, NOT creation time. The commonest trap in the entire Linux section.
- **UID 0** = root, regardless of the account name.
- **/proc** is virtual and exists only in memory — it is not in a dead-box image.
- **ext2 undelete is easier than ext3/ext4** because ext3/4 zero the inode block pointers.
- Hidden files in Linux = a leading dot, **a convention in `ls`, not a file-system attribute**.
- The **sticky bit on a directory** restricts deletion to the file's owner (`/tmp`).

§7. macOS (Mac OS X) Systems and Artifacts

Less likely than Windows in the MCQ, but the syllabus names it explicitly, so a 5-mark question is entirely plausible. Learn the locations table and the four "signature" artifacts:

plists, Keychain, Spotlight and FSEvents.

7.1 Layout and the two library trees

macOS is a **BSD Unix** underneath (Darwin kernel = XNU), so everything in §6 about inodes, permissions, dot-files and `/etc` largely applies. On top sits an Apple-specific layer.

Path	Contents
<code>/Applications</code>	Installed apps (each a .app bundle — actually a directory)
<code>/System</code> , <code>/usr</code> , <code>/bin</code> , <code>/sbin</code> , <code>/etc</code> , <code>/var</code> , <code>/tmp</code>	Unix layer. On modern macOS /System is on a read-only, sealed system volume (SSV) , with user data on a separate Data volume in the same APFS container
<code>/Library</code>	System-wide application support, preferences, launch agents/daemons
<code>/Users/<user>/Library</code>	Per-user equivalent — the richest evidence source. Hidden by default in Finder
<code>/Volumes</code>	Mount points for all mounted volumes (including DMGs and external drives)

<code>/private/var/log</code> , <code>/private/etc</code>	The real locations that <code>/var</code> and <code>/etc</code> symlink to
<code>/.Spotlight-V100</code> , <code>/.fseventsd</code> , <code>/.Trashes</code> , <code>/.DocumentRevisions-V100</code>	Hidden root-level directories — see below

7.2 System startup and services — launchd and plists

- `launchd` is **PID 1** on macOS. It replaced `init`, `cron`, `inetd`, `at` and the old `StartupItems`. **Everything that starts automatically is a `launchd` job** described by a **property list (plist)** file.
- The four `launchd` directories — the macOS persistence answer:**

Path	Runs as	When
<code>/System/Library/LaunchDaemons/</code>	root, no user session	Apple system daemons (do not modify)
<code>/Library/LaunchDaemons/</code>	root, at boot, no user needed	Third-party and malware persistence — the highest-privilege location
<code>/Library/LaunchAgents/</code>	The logged-in user, at login (all users)	Third-party and malware
<code>/Users/<u>/Library/LaunchAgents/</code>	That user, at that user's login	Per-user malware persistence
(legacy) <code>/Library/StartupItems</code> , <code>/System/Library/StartupItems</code>	root	Deprecated
Login Items	Per user	<code>~/Library/Application Support/com.apple.backgroundtaskmanagementagent/BackgroundItems-v*.btm</code> and System Settings → Login Items △ verify current path per macOS version
Also: <code>/etc/periodic/</code> , <code>cron (/usr/lib/cron/tabs/)</code> , <code>/etc/rc.common</code> , kernel extensions (<code>/Library/Extensions</code>), and modern system extensions		

- Property lists (plists)** are Apple's universal configuration format. Two encodings: **XML** (human-readable) and **binary (bplist)** — which starts with the magic bytes

bplist00 and must be converted (`plutil -convert xml1 file.plist`) before reading. **Most modern plists are binary.** Key launchd plist keys: `Label`, `ProgramArguments`, `RunAtLoad`, `StartInterval`, `KeepAlive`, `WatchPaths`.

- Preferences live in `~/Library/Preferences/*.plist` (per app, named `com.vendor.app.plist`) and `/Library/Preferences/`. **Global system settings** are in `/Library/Preferences/SystemConfiguration/`.

7.3 Network configuration

Artifact	Location	Value
Interfaces, DHCP, DNS, proxies, computer name	<code>/Library/Preferences/SystemConfiguration/preferences.plist</code>	The master network configuration
Wi-Fi networks joined — SSIDs, security type, last-connected timestamps, sometimes BSSIDs and channel	<code>/Library/Preferences/com.apple.wifi.known-networks.plist</code> (modern) or <code>/Library/Preferences/SystemConfiguration/com.apple.airport.preferences.plist</code> (older) Δ verify path for the macOS version in question	Geolocation of the machine over time , via BSSID lookup
DHCP leases	<code>/private/var/db/dhcpclient/leases/</code>	IP addresses held and when
Airport/Wi-Fi passwords	Keychain (§7.5)	—
Firewall config	<code>/Library/Preferences/com.apple.alf.plist</code>	Application-layer firewall state
Sharing services (SSH, screen sharing, file sharing)	<code>/Library/Preferences/com.apple.*.plist</code> , launchd daemons	Remote access enabled?
Bluetooth paired devices	<code>/Library/Preferences/com.apple.Bluetooth.plist</code>	Paired phones, keyboards
Network usage per app	<code>/private/var/networkd/netusage.sqlite</code> Δ verify	Bytes in/out per process

7.4 Hidden directories and files

- **The Unix rule applies:** a leading dot hides a file from Finder and `ls`.
- macOS **additionally** has a **hidden file flag** (`chflags hidden`), and Finder honours `/.hidden`.
- Finder's "Show hidden files" is $\text{⌘} + \text{⇧} + \text{.}$
- **The important hidden items:**

Item	Meaning
------	---------

<code>.DS_Store</code>	Created by Finder in every folder the user views , recording icon positions and view options. Forensic gold: it proves a folder was opened in Finder, and it lists the file names that were in that folder — even after those files are deleted. They also leak onto USB drives and web servers
<code>.Trashes</code> / <code>~/Trash</code>	The macOS recycle bin. <code>~/Trash</code> holds deleted items; <code>.Trashes/<uid>/</code> on other volumes. Original paths are tracked in <code>.DS_Store/com.apple.metadata</code> attributes Δ verify mechanism per version
<code>.fsevents</code>	See §7.5
<code>.Spotlight-V100</code>	See §7.5
<code>.DocumentRevisions-V100</code>	Versions — earlier saved versions of documents, retained automatically by the Versions/Auto-Save feature. Recovers the content of documents the user later edited or "cleaned"
<code>._filename</code> (AppleDouble)	Resource-fork/metadata companion files created when copying to non-HFS/APFS media (e.g. a FAT USB stick) — they carry macOS metadata onto foreign file systems
<code>~/Library</code>	Hidden by default in Finder, but the richest per-user evidence store
Extended attributes	<code>xattr -l file</code> ; notably <code>com.apple.quarantine</code> — Apple's Mark-of-the-Web: records that a file was downloaded, by which application, when, and often the source URL . The macOS analogue of <code>Zone.Identifier</code>
<code>com.apple.metadata:kMDItemWhereFroms</code>	An extended attribute holding the download URL and referrer

7.5 System logs and user artifacts

Logs

Log	Location	Note
Unified Logging (macOS 10.12+)	<code>/private/var/db/diagnostics/*.tracev3</code> + <code>/private/var/db/uuidtext/</code>	Binary, compressed, high-volume. Read with <code>log show --predicate ...</code> or forensic parsers (e.g. <code>mandiant/macOS-UnifiedLogs</code>). Replaced most legacy <code>.log</code> files. Retention is short (days to weeks) — collect early
Legacy ASL	<code>/private/var/log/asl/*.asl</code>	Apple System Log — older systems
Classic Unix logs	<code>/private/var/log/</code> — <code>system.log</code> , <code>install.log</code> , <code>wifi.log</code> , <code>secure.log</code> (older)	<code>install.log</code> is very useful: records every software installation and macOS update with timestamps
Audit trail	<code>/private/var/audit/</code>	BSM audit records, if <code>auditd</code> enabled

Crash/ diagnostic reports	<code>~/Library/Logs/ DiagnosticReports/</code> , <code>/Library/ Logs/DiagnosticReports/</code>	Proves a program ran (it crashed) and often captures its arguments and loaded libraries
Console/user logs	<code>~/Library/Logs/</code>	Per-application logs

The four signature user artifacts

Artifact	Location	What it gives you
Keychain	<code>~/Library/Keychains/login.keychain-db</code> (user), <code>/Library/Keychains/System.keychain</code> (system, incl. Wi-Fi passwords), <code>~/Library/Keychains/<UUID>/keychain-2.db</code> (iCloud/data-protection keychain)	Stored passwords — website logins, Wi-Fi, app credentials, certificates, secure notes. Encrypted with the user's login password ; if you have the password (or the user's session is live) the contents are extractable (<code>security dump-keychain -d</code>). Even without the password, the metadata — service names, account names, creation and modification dates — reveals which accounts and services the user held
Spotlight	<code>/.Spotlight-V100/Store-V2/<UUID>/store.db</code> (and <code>.store.db</code>); per-volume, including external drives	The system-wide metadata index of every file — names, paths, sizes, MIME types, and content-derived metadata , plus <code>kMDItemUsedDates</code> / last-used and last-download dates . Crucially it retains entries for files that have been deleted , letting you prove a document with a given name and size existed. Query a live system with <code>mdfind</code> , inspect a file's metadata with <code>mdls</code>
FSEvents	<code>/.fseventsd/</code> (per volume, including external and USB volumes)	The file-system change journal used by Time Machine and Spotlight. Records that a path was created, modified, renamed or deleted , with an event ID ordering. It gives you a history of file-system activity including on removable drives — often the only proof that files were copied to a USB device . Note: it stores paths and event flags, not timestamps for every event , so times are approximate/derived from the log-file boundaries
Recents / recent items	<code>~/Library/Application Support/com.apple.sharedfilelist/*.sfl2</code> (Recent Documents, Recent Applications, Recent Servers, Recent Volumes), plus per-app <code>com.apple.recentitems</code> plists	The macOS analogue of Windows RecentDocs/Jump Lists. RecentServers and RecentVolumes prove connections to network shares and external drives

Other high-value macOS user artifacts

Artifact	Location

Time Machine backups	<code>/Volumes/<backup>/Backups.backupdb/</code> or APFS snapshots — complete historical copies of the whole system
APFS local snapshots	<code>tmutil listlocalsnapshots /</code> — recoverable prior states even without an external backup drive
Safari history / downloads / bookmarks / cache	<code>~/Library/Safari/History.db</code> , <code>Downloads.plist</code> , <code>Bookmarks.plist</code> , <code>~/Library/Caches/com.apple.Safari/</code> — see FOR_03 §8
Messages / iMessage	<code>~/Library/Messages/chat.db</code> (SQLite) + <code>Attachments/</code> — often the single most probative file on a Mac
Mail	<code>~/Library/Mail/V*/</code> — <code>.emlx</code> message files
Notes, Calendar, Contacts, Reminders	<code>~/Library/Group Containers/...</code> (SQLite)
Photos library	<code>~/Pictures/Photos Library.photoslibrary/database/Photos.sqlite</code>
Quarantine event database	<code>~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2</code> (SQLite) — every file downloaded, with the URL and timestamp △ verify path on recent versions
Installed apps and their first-run	<code>/var/db/SystemPolicyConfiguration/ExecPolicy</code> (SQLite), <code>install.log</code>
Terminal history	<code>~/.bash_history</code> , <code>~/.zsh_history</code> (zsh is the default shell since Catalina)
iOS device backups	<code>~/Library/Application Support/MobileSync/Backup/</code> — a full iPhone backup sitting on the Mac . Always check for this in a mobile case
User account records	<code>/private/var/db/dslocal/nodes/Default/users/*.plist</code> (the macOS equivalent of <code>/etc/passwd+shadow</code> — includes the <code>ShadowHashData</code> password verifier)

§7.6 Likely exam questions (macOS)

Marks	Question
5	What is a plist file? Where are plists found on macOS?
5	What is <code>launchd</code> ? Name the directories used for automatic startup.
5	Write a note on the macOS Keychain from a forensic point of view.
5	What are FSEvents and why are they forensically important?
10	Describe the principal user artifacts on a macOS system and their locations.
10	Compare Windows, Linux and macOS with respect to hidden files, startup persistence and system logging.
15	Discuss macOS system and user artifacts under the headings: startup and services, network configuration, hidden directories, system logs and user artifacts.

MCQ traps (macOS)

- **launchd** is PID 1 on macOS (not `init`, not `systemd`).
- Binary plists start with `bplist00` and must be converted before reading.
- `.DS_Store` records folder view settings and proves a folder was opened in Finder.
- Spotlight index = `.Spotlight-V100`; FSEvents = `.fsevents.d`. Do not swap.
- `com.apple.quarantine` is the macOS equivalent of the NTFS `Zone.Identifier` stream.
- APFS is copy-on-write with native snapshots; HFS+ is not.
- The default shell on modern macOS is **zsh**, not bash.

§8. Cross-platform artifact comparison — one table for revision

Question	Windows	Linux	macOS
Config store	Registry (<code>SYSTEM</code> , <code>SOFTWARE</code> , <code>SAM</code> , <code>NTUSER.DAT</code>)	Text files in <code>/etc</code> and dot-files in <code>~</code>	plists in <code>/Library</code> and <code>~/Library</code>
Init / PID 1	<code>wininit/services.exe</code>	<code>init</code> / <code>systemd</code>	<code>launchd</code>
Persistence	Run keys, Services, Scheduled Tasks, Startup folder, WMI	<code>cron</code> , <code>systemd timers/</code> <code>services</code> , <code>rc.local</code> , <code>.bashrc</code> , <code>/etc/init.d</code>	LaunchDaemons / LaunchAgents , Login Items
Authentication log	<code>Security.evtx</code> (4624/4625)	<code>auth.log</code> / <code>secure</code> , <code>wtm</code> , <code>btm</code>	Unified log, <code>/var/log/system.log</code> , ASL
Hidden files	H/S attributes + ADS	Leading dot (a convention)	Leading dot + hidden flag + <code>~/Library</code> hidden by Finder
"Downloaded from the internet" marker	<code>:Zone.Identifier</code> ADS	(none standard)	<code>com.apple.quarantine</code> xattr + QuarantineEvents DB
Recycle bin	<code>C:\\$Recycle.Bin\<SID>\\$I/\$R</code>	<code>~/ .local/share/Trash/{files,info}</code>	<code>~/ .Trash</code> , <code>/ .Trashes/<uid></code>
Recent files	RecentDocs, LNK, Jump Lists	<code>.recently-used.xbel</code> , <code>.viminfo</code>	<code>*.sfl2</code> shared file lists
Folder-browsing proof	ShellBags	(weak — mount logs, shell history)	<code>.DS_Store</code>
Program execution proof	Prefetch , UserAssist, ShimCache, Amcache, BAM	<code>auth.log/audit.log</code> , <code>.bash_history</code> , package logs	Unified log, <code>ExecPolicy</code> , crash reports, quarantine
Thumbnail cache	<code>thumbcache_*.db</code>	<code>~/ .cache/thumbnails/</code>	Quick Look thumbnails, Photos DB

Parity, checksums vs cryptographic hashes	A checksum (CRC32) detects accidental error; a cryptographic hash (SHA-256) resists deliberate forgery. Do not confuse them — see FOR_03 §5.4.
Boolean logic in search	Keyword searches and filters in EnCase/FTK use Boolean and regular-expression logic.

9.2 TP-I Unit 8 — Operating Systems, UNIX, Windows OS

→ Covered in [_Shared\CORE_03_OperatingSystems.md](#) and [_Shared\CORE_04_UnixLinux.md](#) — revise those.

(OS functions; multiprogramming/multiprocessing/multitasking; virtual memory, paging, fragmentation; mutual exclusion, critical regions, locks; CPU/IO/resource scheduling, deadlock; disk management, formatting, boot block, free-space management, RAID; protection and security, domains, access control, Trojan/virus/worm, authentication; the UNIX file system, process management, shell variables, command-line programming, filters and commands; Windows design principles, system components, Terminal Services, Fast User Switching, file system, networking.)

Plus these forensic-specific angles (most are developed in detail in §5 and §6 above):

OS concept	Forensic slant
File systems	§4, §5.9 — allocation units, file slack , unallocated space, journaling as an evidence source
Deleted-file mechanics	§4.2, §6.2 — deletion changes bookkeeping, not data. FOR_03 §6 for recovery and carving
Virtual memory / paging	pagefile.sys and the swap partition are memory on disk — searchable for keys, passwords and document fragments (§5.12)
Hibernation	hiberfil.sys is a full RAM image on disk — the best offline memory evidence (§5.12)
Process management	Live forensics: process list, parent-child relationships (a <code>winword.exe</code> spawning <code>cmd.exe</code> is anomalous), loaded DLLs, handles, network sockets. Volatility does all of this from a memory image
Scheduling / services	Persistence mechanisms — Services, Scheduled Tasks, cron, systemd timers (§5.3d, §6.8)
Free-space management	The bitmap/FAT is <i>exactly</i> the structure that tells you which clusters are unallocated → the carving target
RAID	Must image every member disk and reconstruct; note the level, stripe size and disk order (§2.3)
Protection, access control, authentication	ACLs and permissions establish who could have done it ; SAM/shadow hashes establish credential compromise ; SUID files are backdoors (§6.3)
Trojan / virus / worm	Malware taxonomy is examined in FOR_03 §2.7

UNIX commands	The examiner's own toolset: <code>dd</code> , <code>dc3dd</code> , <code>md5sum/sha256sum</code> , <code>strings</code> , <code>grep -a</code> , <code>file</code> , <code>xxd/hexdump</code> , <code>stat</code> , <code>lsof</code> , <code>find -perm -4000</code> , <code>last</code> , <code>lastb</code> , <code>journalctl</code> , <code>mount -o ro,noexec,nodev,noatime,loop</code> and <code>losetup -r</code> for read-only image mounting, <code>fdisk -l</code> , <code>blkid</code> , <code>debugfs</code> , <code>foremost/scalpel</code> for carving
Windows Terminal Services / Fast User Switching	Multiple concurrent user sessions → attribution problem : which session did the act? Resolve with logon type 10 (RDP) events, session IDs, and per-user artifacts
Windows networking	SMB share access events (5140/5145), <code>MountPoints2</code> , mapped-drive MRU

9.3 TP-I Unit 9 — Computer Organization & Architecture

→ Covered in `_Shared\CORE_02_ComputerOrganisation.md` — revise that.

(Basic structures and operational concepts; instruction formats, execution and sequencing; addressing modes; stacks, queues, subroutines; register transfers; RAM/ROM types; cache; performance, interleaving, hit rate; memory hierarchy and virtual memory address translation; secondary memory; I/O organisation, memory-mapped vs isolated I/O; programmed I/O, interrupts, DMA; synchronous and asynchronous buses and standard interfaces.)

Plus these forensic-specific angles:

Architecture concept	Forensic slant
Memory hierarchy	Directly generates the order of volatility (FOR_03 §4.3): registers/cache → RAM → swap → disk → archival
DMA	DMA over FireWire/Thunderbolt/PCIe was historically used to read a locked machine's RAM directly ("Inception"-style attacks) — a live-acquisition technique and a security risk. Modern IOMMU/Kernel DMA Protection mitigates it
Cache and interleaving	Explains why registers/cache are <i>practically</i> unrecoverable — nothing can freeze them
Standard interface buses	The examiner's practical world: SATA, PATA/IDE, NVMe/M.2, SAS, USB 2/3, Thunderbolt, FireWire, eSATA, SCSI . You must know which write blocker or adapter each needs. NVMe drives require an NVMe-capable blocker — a common practical trap
ROM / firmware	BIOS/UEFI implants; firmware-level malware survives disk wiping and OS reinstallation
TPM	Stores BitLocker keys ; a disk removed from its motherboard may be unrecoverable without the recovery key, because the TPM will not release the key to different hardware. Always search for the BitLocker recovery key (printed, in a Microsoft account, or in AD) before removing the drive
Instruction sets and addressing	Needed for reverse engineering and malware analysis (disassembly, stack frames, calling conventions)
I/O organisation	Printers, scanners and MFDs have their own storage and job logs; a networked MFD can be an evidence source

Appendix — One-page revision sheet for FOR_02

Disk geometry: platter → track → **cylinder** (same track, all surfaces) → **sector** (512 B / 4 KB, smallest *disk* unit) → **cluster** (smallest *file-system* unit). CHS → **LBA**. Hidden:

HPA, DCO, remapped sectors.

SSD problem: wear levelling + **TRIM** + garbage collection ⇒ deleted data erased autonomously; **two images of the same untouched SSD may hash differently**; carving unreliable. Over-provisioning is unreachable without chip-off.

Boot: POWER → **POST** → BIOS/UEFI → (**MBR** at LBA 0, 446 boot + 64 partition table + **55AA** | **UEFI** → **ESP** **.efi**) → bootloader (BOOTMGR/GRUB) → kernel → SMSS/CSRSS/WINLOGON or systemd → logon.

File systems: FAT32 — max file **4 GB-1**, deletion sets first name byte **0xE5** and frees the chain. NTFS — **MFT** (1 KB records; **\$SI** vs **\$FN** timestamps; resident < ~700 B), **\$LogFile**, **\$UsnJrnl**, **\$Bitmap**, **ADS**, VSS. ext2 easy undelete; **ext3/4 zero the inode pointers**. APFS = CoW + snapshots + encryption.

Registry hives: `System32\config\` → **SYSTEM, SOFTWARE, SAM, SECURITY, DEFAULT**; `C:\Users\<u>\` → **NTUSER.DAT**; `AppData\Local\Microsoft\Windows\` → **UsrClass.dat** (**ShellBags**). HKLM\HARDWARE is **RAM-only**. **Keys have LastWrite; values do not**.

Key registry artifacts: **USBSTOR** (serial) + **MountedDevices** (GUID) + **MountPoints2** (user) + **setupapi.dev.log** (first connect) = the USB chain. **UserAssist** = **ROT-13, GUI execution**. **ShellBags** = **folder browsed, survives deletion**. MRUs: RecentDocs, RunMRU, TypedPaths, **WordWheelQuery** (search terms), OpenSave/LastVisitedPidIMRU. **Run/RunOnce, Winlogon Shell/Userinit, IFE0 Debugger, Services** = persistence.

Execution evidence: **Prefetch** (`C:\Windows\Prefetch\*.pf`, run count, **last 8 run times on Win8+**, 1024 entries, path hash in the name) · **UserAssist** · **ShimCache** = **presence** · **Amcache** = **SHA-1 hash** · **BAM** · **SRUM** (**network bytes**) · **Event 4688**.

Event IDs: **4624** logon (type **2** console, **3** network, **10** RDP) · **4625** failed · 4648 explicit creds · 4672 admin · 4720 account created · **4688** process created · **1102 security log cleared** · **7045 new service** · 6005/6006 boot/shutdown · **6008 unexpected shutdown**.

LNK: target path + target MAC times + size + **volume serial and label** + **machine NetBIOS name and MAC**. Survives deletion of the target and removal of the drive.

ADS: `file:stream:$DATA`; NTFS-only; invisible to Explorer, size not counted; detect with `dir /R / Get-Item -Stream * / streams.exe`; **:Zone.Identifier** = **proof of download**; destroyed by copying to FAT.

Slack: **RAM slack** = to end of the **sector** (now usually zero-padded) · **drive slack** = remaining unused **sectors** of the last **cluster** · **file slack** = **RAM + drive slack** · **volume slack** = **end of file system** → **end of partition** · **unallocated** = **whole free clusters (deleted file bodies)** · unpartitioned = outside every partition. **Only a physical bit-stream image captures all of these**.

Recycle Bin: `C:\$Recycle.Bin\<SID>\` → **\$I** = **metadata (original path, size, deletion time)**, **\$R** = **content**. XP: **RECYCLER** + **INFO2**. Shift+Delete never enters it.

Memory on disk: `hiberfil.sys` = compressed RAM image (analyse with Volatility) ·

`pagefile.sys` / `swapfile.sys` = paged-out memory (keyword search) · `MEMORY.DMP` ·

`thumbcache_* .db` = images that no longer exist.

Linux: `inode` = all metadata + block pointers; name lives in the **directory**.

`atime` / `mtime` / `ctime` (= `inode change`, not creation) / `ctime` (`ext4`); `ctime` not user-settable

→ **timestamping detector**. `/etc/passwd` **7 fields** (`x` → shadow; **UID 0 = root**),

`/etc/shadow` **9 fields** (`$6$` = SHA-512), `/etc/group` **4 fields**. **SUID 4000** —

`find / -perm -4000`. Dot-file = convention, not an attribute. Logs: `auth.log`/`secure`,

`syslog`/`messages`, `wtmp` (**last**) / `btmpt` (**lastb**) / `lastlog` / `utmp` (**who**), `journalctl`,

`audit.log`, `apt/dpkg/yum` logs. `.bash_history` (written at clean exit; no times unless

`HISTTIMEFORMAT`), `.viminfo`, `.mysql_history`, `.ssh/authorized_keys` and `known_hosts`.

Cron: `/etc/crontab`, `/etc/cron.d`, `/var/spool/cron/crontabs/<user>`, **systemd timers**.

macOS: `launchd` = PID 1; persistence in `/Library/LaunchDaemons` (root),

`/Library/LaunchAgents`, `~/Library/LaunchAgents`. **plist**s (binary ones start `bplist00`).

`.DS_Store` = folder opened in Finder. `.Spotlight-V100` = metadata index incl. deleted

files (`mdls`, `mdfind`). `.fsevents` = file-system change journal, **per volume including**

USB. **Keychain** = stored passwords (`login.keychain-db`, `System.keychain`).

`com.apple.quarantine xattr` = downloaded-from marker. `~/Library` is the evidence store.

Unified logs in `/private/var/db/diagnostics/*.tracev3`, short retention.

`~/Library/Application Support/MobileSync/Backup/` = **iPhone backups on the Mac**.